

Chapter 4: Registration and Normalization

Prof. Dr.-Ing. Thomas Schultz

URL: <http://cg.cs.uni-bonn.de/schultz/>

E-Mail: schultz@cs.uni-bonn.de

Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

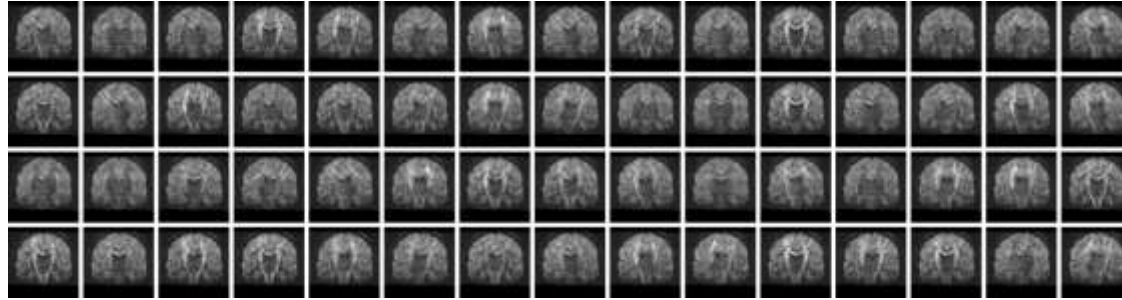
November 6/10/13/20, 2025

Image Acquisition and Analysis in Neuroscience

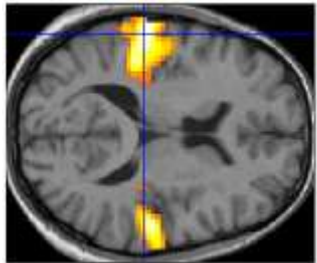
Acquisition



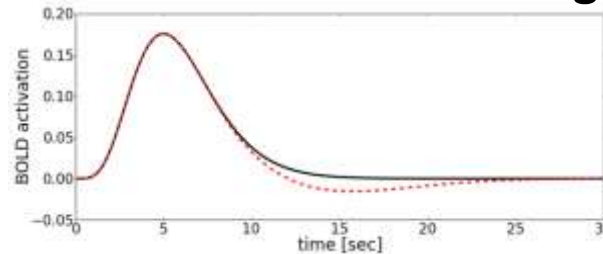
Reconstruction & Artifact Correction



Hypothesis Testing



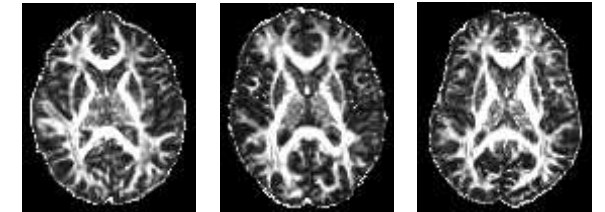
Mathematical Modeling



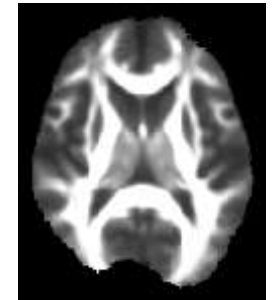
Segmentation



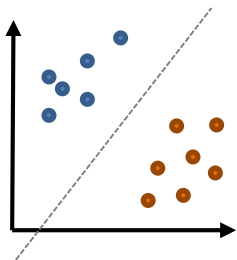
Registration



Atlas Construction



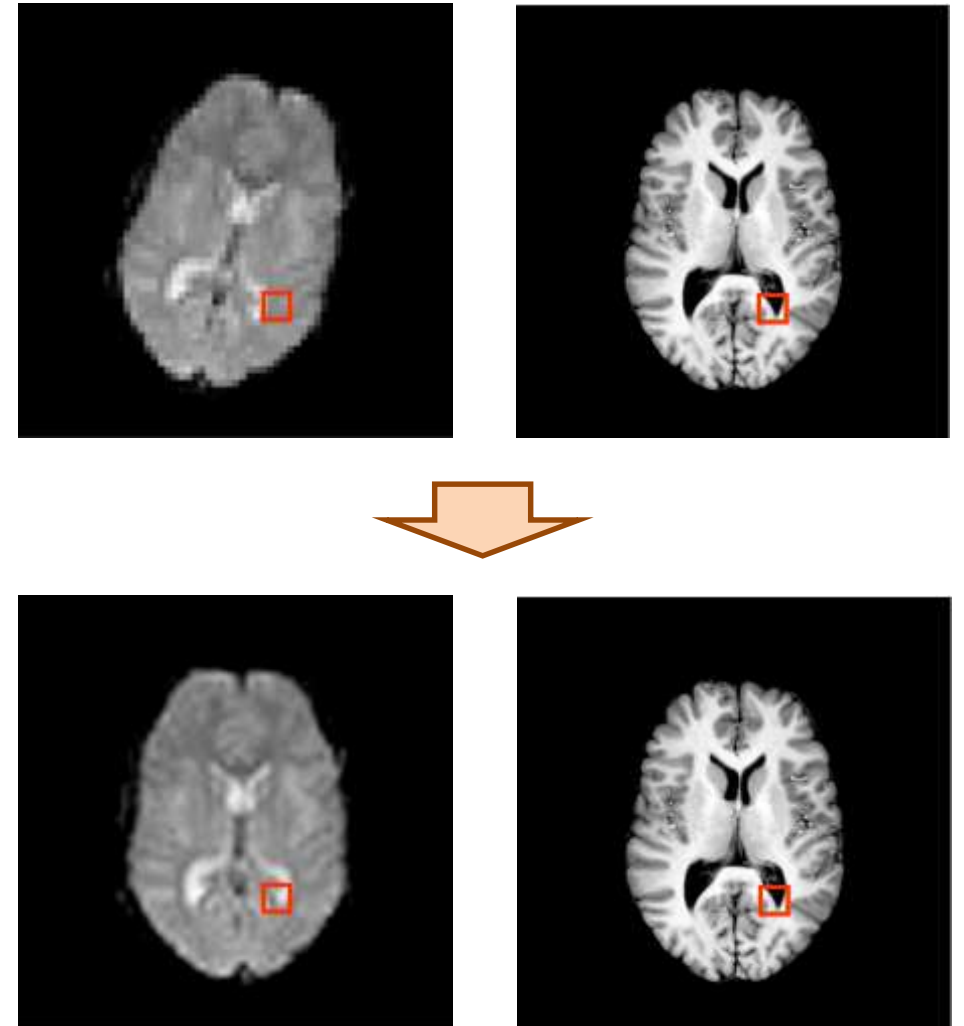
Machine Learning



4.1 Image Registration: Goals and Definitions

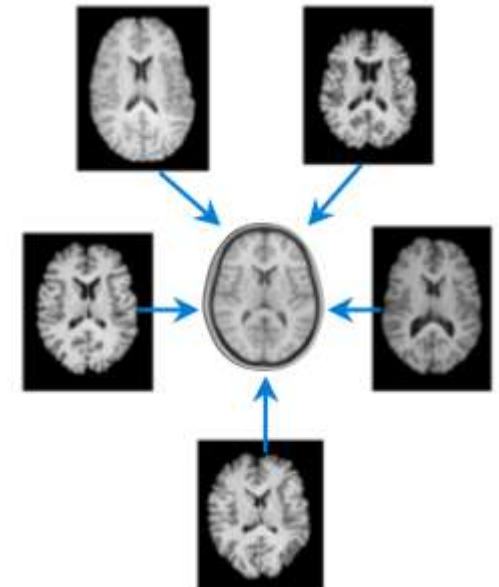
Registration: Problem Statement

- **Registration** brings different images into spatial alignment
 - The same voxel position should correspond to the same anatomical location



Registration: Use Cases

- **Example uses of registration in neuroimaging:**
 - Same subject and modality **along time**
 - *Example* within-session: Compensating head motion
 - *Example* between-session: Studying neuronal plasticity
 - Fusing images from **different modalities**
 - *Example:* Identify brain region in structural MRI, detect activations with fMRI; compensate for EPI distortions
 - Comparing **different subjects**
 - Based on comparing image intensity
Example: Tissue integrity is affected by some disease.
 - Based on the spatial mapping itself
Example: The hippocampus is enlarged in some population.

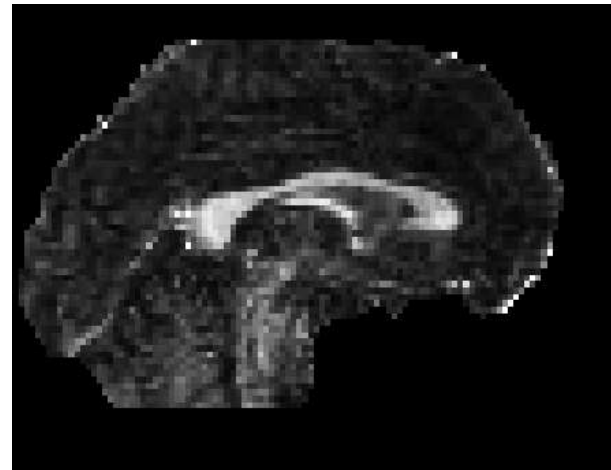


Registration: Terminology

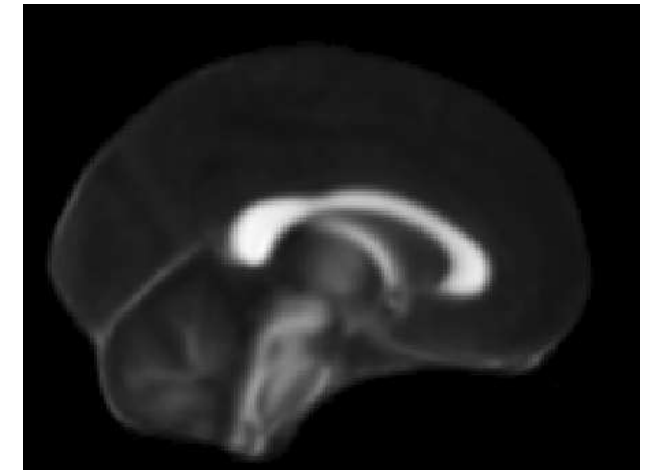
- Registration involves (at least) two images. Most algorithms differentiate
 - **Reference / fixed image:** Remains unchanged throughout the registration process
 - **Floating / moving image:** Is transformed to match the reference
- Registration of images from different subjects is often called **normalization**

Linear vs. Non-Linear Registration

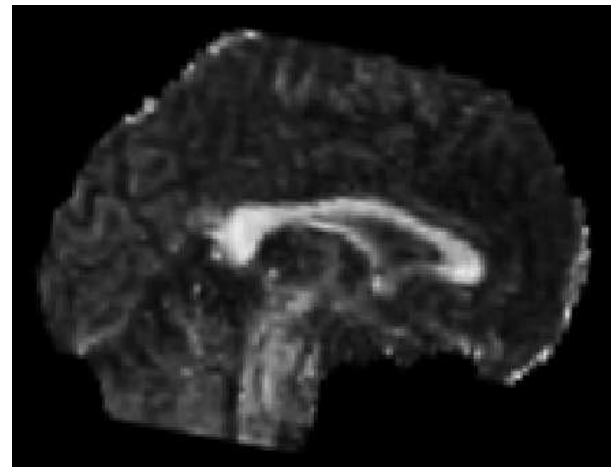
- **Affine („linear“)** registration only permits a few global transformations
 - Rotation, translation, scaling, shear
- **Nonlinear** registration permits local deformations
 - Required to align different subjects
 - Typically performed as a refinement after linear registration



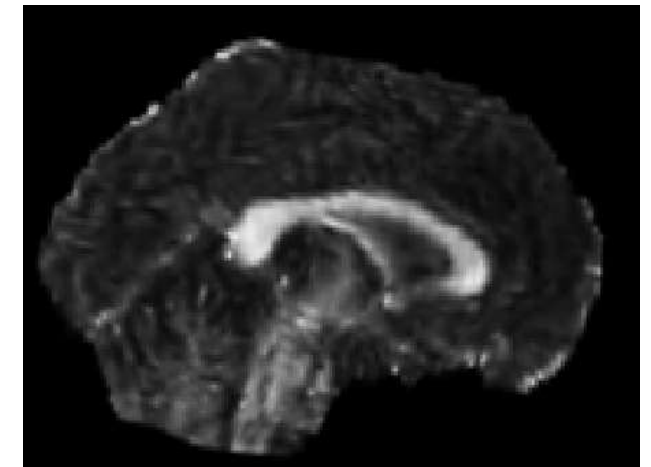
Single Subject Scan



Template



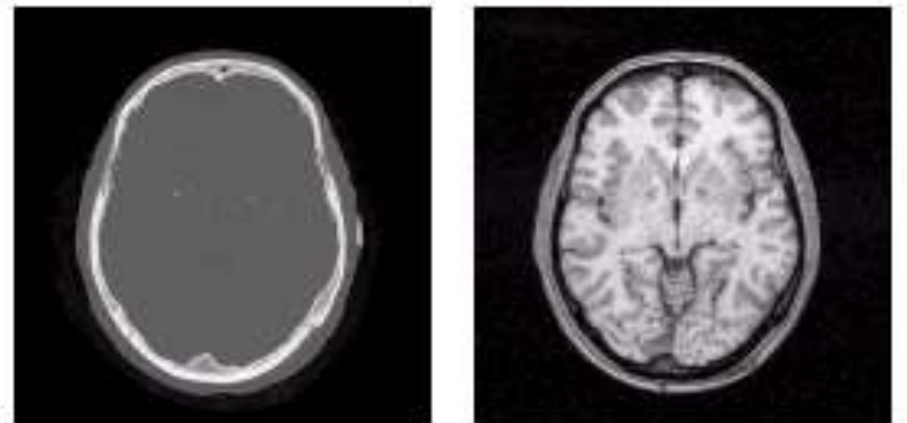
Affine Transformation



Nonlinear Warp

Intra- vs. Inter-Modality Registration

- **Intra-modality** (within-modality) registration
 - When comparing images from the same modality (e.g., MRI) and sequence, it makes sense to directly compare image intensity
 - *Example:* Motion correction in fMRI
 - May have to adjust global brightness
- **Inter-modality** (between-modality) registration
 - Same object can have different intensity in different modalities or between MR sequences

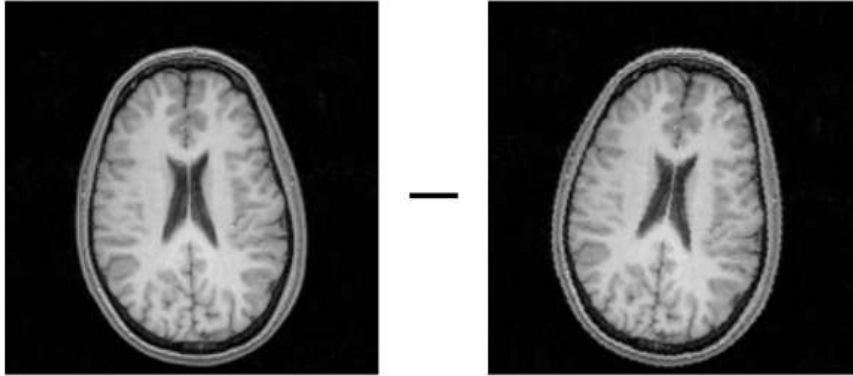


Example: CT vs. MRI

Optimization-based Registration

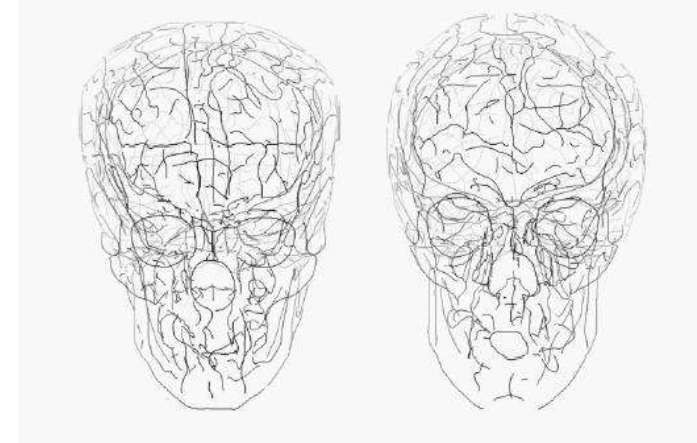
- Most traditional registration algorithms solve an **optimization problem** (possibly implicitly)
 - Iteratively optimize over some space of transformations
 - Cost function quantifies dissimilarity between images based on

Image intensity in each voxel



Main strategy in neuroimaging and this lecture

Proximity of feature points / curves



- Cost often also accounts for the irregularity of the applied transformation, goal is to prefer less extreme deformations if possible

Regression-based Registration

- **Regression-based approaches** directly map two misaligned images / image patches to transformation parameters
 - Based on deep neural networks which are trained using the same principles as optimization based methods
 - *Main benefit:* Much faster once training has completed
 - In example on the right: 35x

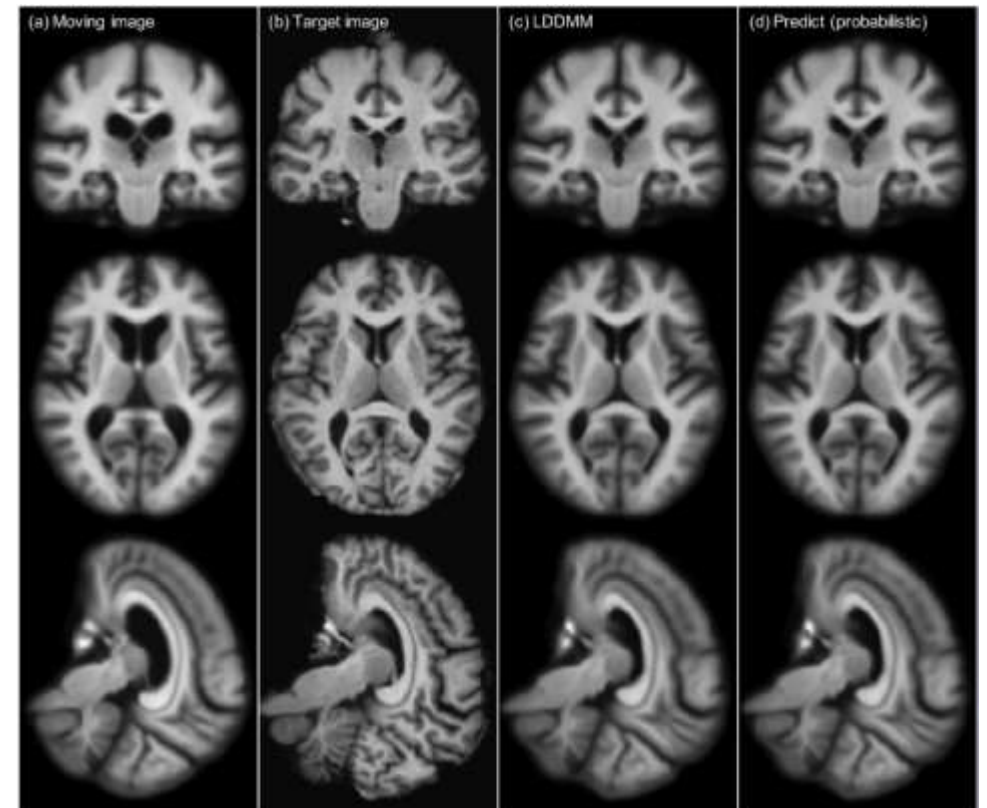


Image from: Xiao Yang et al., „Quicksilver: Fast predictive image registration – A deep learning approach“ NeuroImage 158:378-396, 2017

Landmark based Evaluation

Based on the positions of certain anatomical **landmarks**

- $\mathbf{r}_i$ ($i = 1, \dots, m$) in the reference image
- $\mathbf{p}_i$ ($i = 1, \dots, m$) in the moving image

the mean / maximal

Target Registration Error

$$\text{TRE}_{\text{mean}} = \frac{1}{m} \sum_{i=1}^m \|\mathbf{r}_i - T(\mathbf{p}_i)\|$$

quantify the discrepancy $\text{TRE}_{\text{max}} = \max\{\|\mathbf{r}_i - T(\mathbf{p}_i)\| \mid i = 1, \dots, m\}$

that remains after applying the transformation T that has been found by the registration

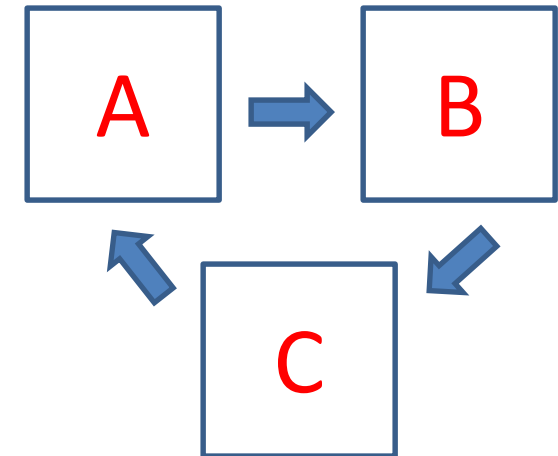
- Landmark positions might be annotated manually, or result from applying a known transformation to test our registration method

Consistency based Evaluation

- Even without landmarks, we can evaluate **consistency**:
 - Does exchanging the roles of reference and moving images produce the inverse transformation?



- Does composition of cyclic transformations produce (almost) the identity?
- Do we get a similar result after adding artificial noise or artifacts to the images?
- Do we get a similar result for an immediate re-scan of the same subject?



Evaluation: Visual Assessment

- **Visual Comparison:**

- Difference / average images
- Feature alignment
- Checkerboard patterns
- Color coding

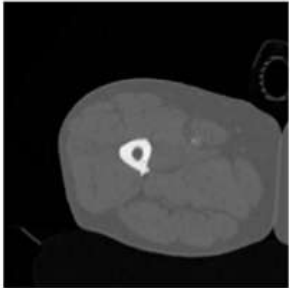


Fig 3.102: CT of a leg

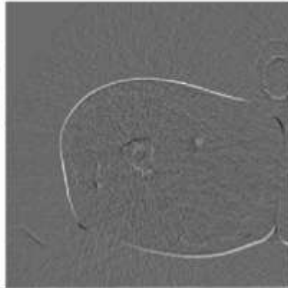


Fig 3.103: Pre-registration image subtraction

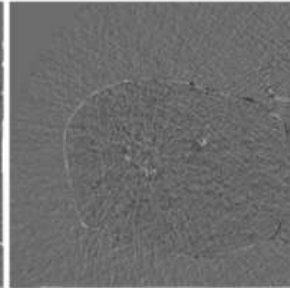
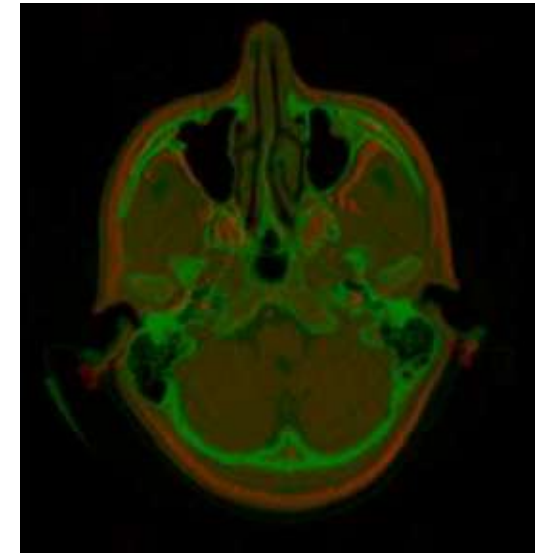
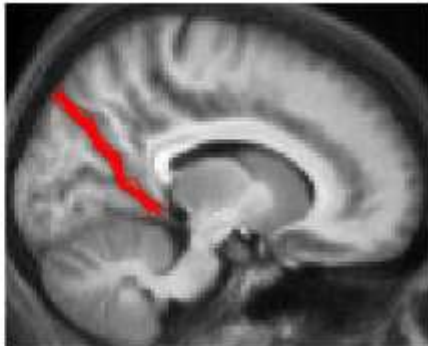
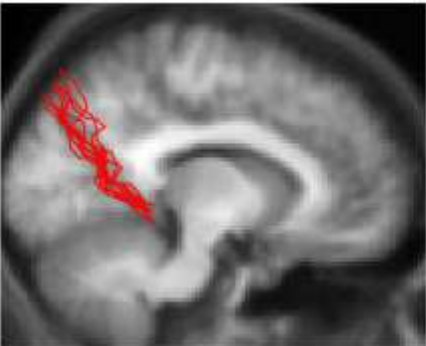


Fig 3.104: Post-registration image subtraction



Ingredients of Intensity-based Registration

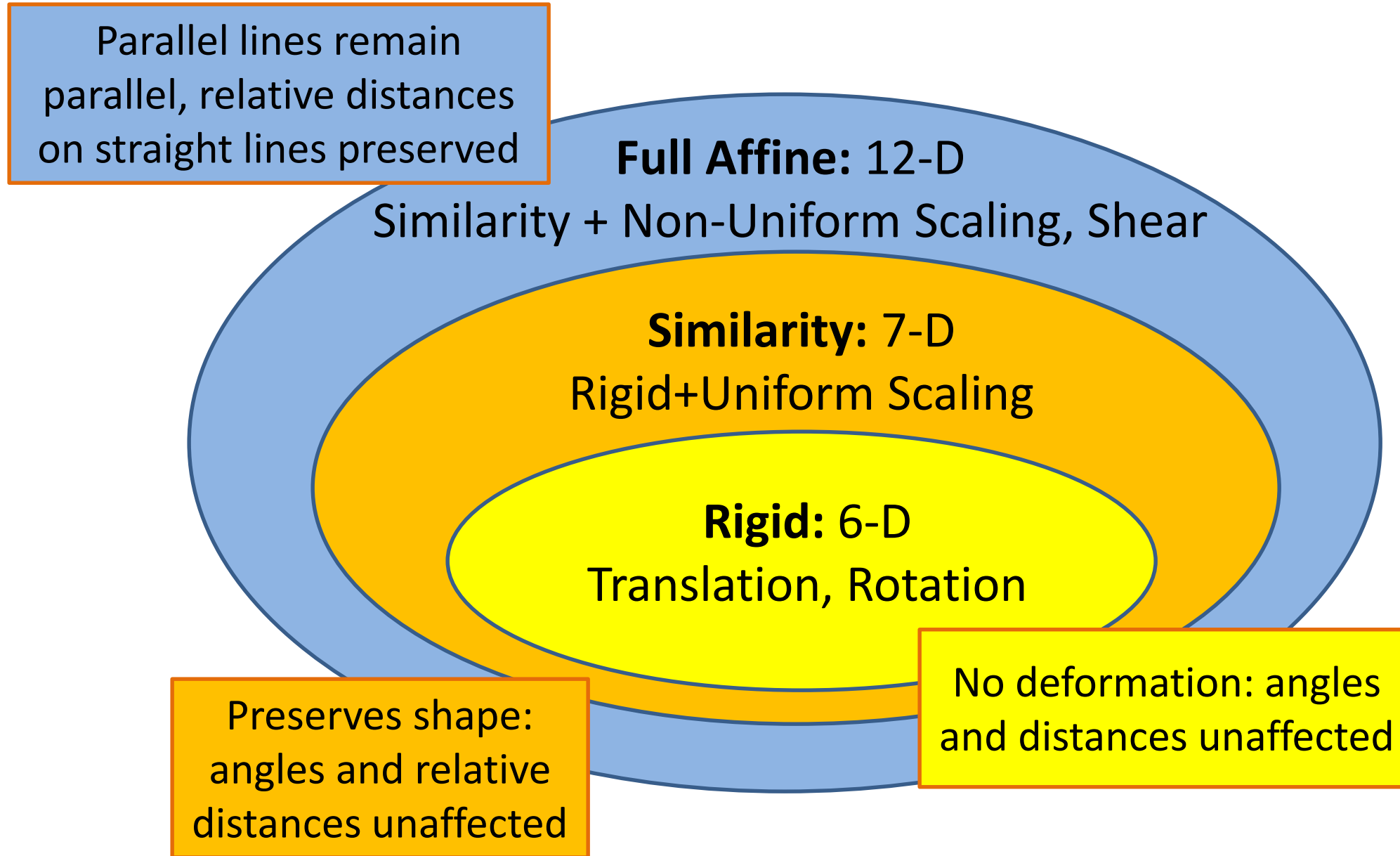
Typical ingredients of **optimization-based registration** with an **intensity-based cost function** are:

- **Search space:** Set of all transformations that we are willing to apply to the floating image
- **Interpolation scheme:** See Chapter 2.3. Consider using
 - fast interpolation (e.g., trilinear) during optimization
 - higher-quality interpolation (e.g., tricubic) to obtain the final result
- **Cost Function:** Should be minimized to achieve registration
- **Optimization method:** Strategy to find a transformation that minimizes cost / maximizes similarity

Evaluation uses landmarks, consistency, and/or visualization

4.2 Affine Transformations

Hierarchy of Affine Transformations



Linear Transformations

Most **fundamental geometric transformations** (scaling, rotation etc.) are linear. Translation is not, but becomes linear when using homogeneous coordinates.

- A function T that maps one vector $\mathbf{x}$ to another vector $\mathbf{y}$ is a **linear transformation** if
 - T is $T(\mathbf{v}+\mathbf{w})=T(\mathbf{v})+T(\mathbf{w})$
 - $T(\alpha\mathbf{v})= \alpha T(\mathbf{v})$
- Matrix-vector multiplication: $\mathbf{y}=T(\mathbf{x})=\mathbf{T}\mathbf{x}$

Elementary Linear Transformations

- **Identity transform:** $\mathbf{Iv}=\mathbf{v}$ for all vectors $\mathbf{v}$
- **Projection** onto span of unit vector $\hat{\mathbf{x}}$: $\mathbf{P}_{\hat{\mathbf{x}}} = \hat{\mathbf{x}}\hat{\mathbf{x}}^T$

$$\hat{\mathbf{x}} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \Rightarrow \mathbf{P}_{\hat{\mathbf{x}}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- **Projection** onto perpendicular space: $\mathbf{P}_{\hat{\mathbf{x}}}^{\perp} = \mathbf{I} - \hat{\mathbf{x}}\hat{\mathbf{x}}^T$

$$\hat{\mathbf{x}} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \Rightarrow \mathbf{P}_{\hat{\mathbf{x}}}^{\perp} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Uniform and Non-Uniform Scaling

- **Uniform scaling** by factor α : $\alpha \mathbf{I}$



- **Scaling** along a unit vector $\hat{\mathbf{x}}$:

- Based on decomposition

$$\mathbf{v} = \mathbf{I}\mathbf{v} = \mathbf{P}_{\hat{\mathbf{x}}}\mathbf{v} + \mathbf{P}_{\hat{\mathbf{x}}}^{\perp}\mathbf{v}$$

- Resulting equation for scaling by factor α along $\hat{\mathbf{x}}$:

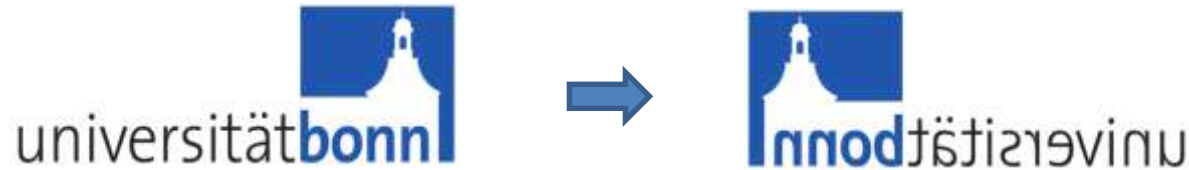
$$\mathbf{S}_{\alpha, \hat{\mathbf{x}}} = \alpha \mathbf{P}_{\hat{\mathbf{x}}} + \mathbf{P}_{\hat{\mathbf{x}}}^{\perp}$$



Reflection and Shear

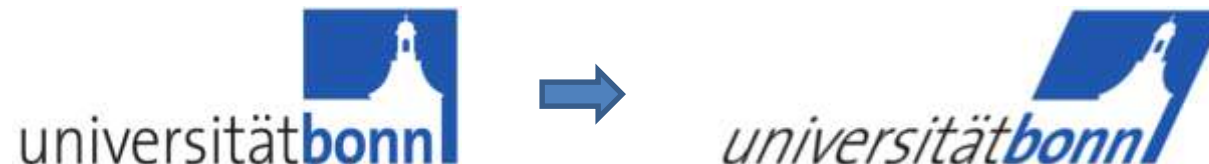
- **Reflection** amounts to non-uniform scaling with $\alpha = -1$:

$$\mathbf{M}_{\hat{\mathbf{x}}} = \mathbf{P}_{\hat{\mathbf{x}}}^{\perp} - \mathbf{P}_{\hat{\mathbf{x}}}$$



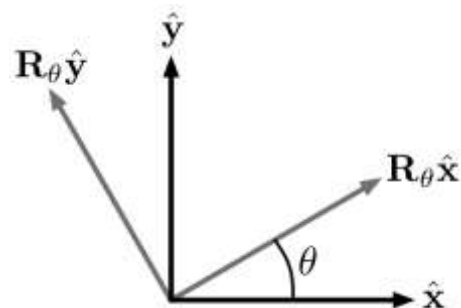
- **Shear** (transvection): Parallel lines are shifted by an amount that is proportional (with shearing factor β) to the (signed) distance along a perpendicular vector

$$\mathbf{T} = \mathbf{I} + \beta \hat{\mathbf{x}} \hat{\mathbf{y}}^T$$



Rotation Matrix

- Within plane $\langle \{\hat{\mathbf{x}}, \hat{\mathbf{y}}\} \rangle$, rotation of $\hat{\mathbf{x}}$ towards $\hat{\mathbf{y}}$ by angle θ is achieved by the following rotation matrix:



$$\mathbf{R}_\theta \hat{\mathbf{x}} = \cos \theta \hat{\mathbf{x}} + \sin \theta \hat{\mathbf{y}}$$

$$\mathbf{R}_\theta \hat{\mathbf{y}} = -\sin \theta \hat{\mathbf{x}} + \cos \theta \hat{\mathbf{y}}$$

$$\mathbf{R}_\theta \mathbf{a} = \mathbf{a}.$$

$$\mathbf{R}_\theta = \mathbf{I} + (\cos \theta - 1)(\hat{\mathbf{x}}\hat{\mathbf{x}}^T + \hat{\mathbf{y}}\hat{\mathbf{y}}^T) + \sin \theta (\hat{\mathbf{y}}\hat{\mathbf{x}}^T - \hat{\mathbf{x}}\hat{\mathbf{y}}^T)$$

- Example with $\hat{\mathbf{x}} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \hat{\mathbf{y}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} : \mathbf{R}_\theta = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$



Translations and Affine Transformations

- **Translation** $f(x)=x+t$ is a fundamental operation, but cannot be represented as a matrix-vector product in the original space
 - Requires moving from linear to **affine** maps: $f(x)=Mx+t$
- If we increase the dimension of x by using 1 as the additional coefficient, the translation can be included in an **affine transformation matrix**:

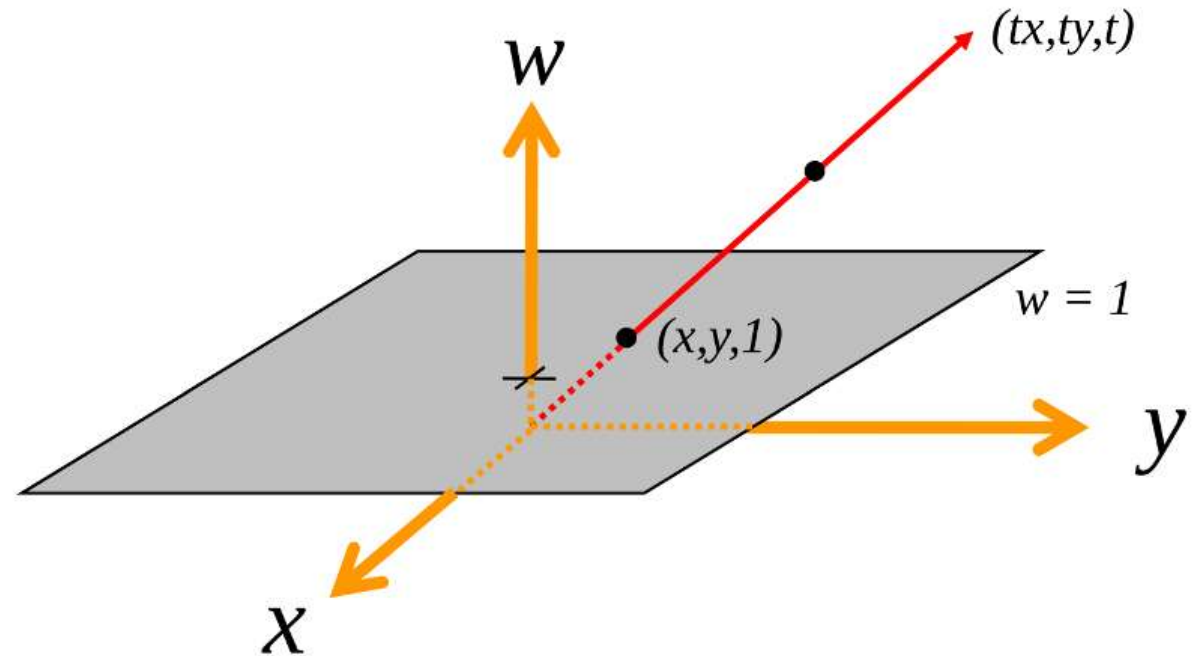
The diagram shows an affine transformation matrix and a vector. The matrix is a 4x4 matrix with elements a, b, c, tx in the first row; d, e, f, ty in the second row; g, h, i, tz in the third row; and $0, 0, 0, 1$ in the fourth row. To the left of the matrix, the text "Scaling + Rotation + Shear" is written, with a curved arrow pointing from this text to the first three columns of the matrix. To the right of the matrix, the text "Translation" is written, with a curved arrow pointing from this text to the fourth column of the matrix. The vector is a column vector with elements $x, y, z, 1$. A bracket is placed above the first three columns of the matrix, and another bracket is placed above the fourth column of the matrix.

$$\begin{pmatrix} a & b & c & tx \\ d & e & f & ty \\ g & h & i & tz \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

- Terms “linear” and “affine registration” are used interchangeably

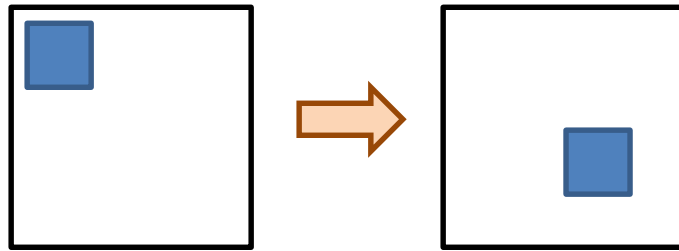
Homogeneous Coordinates

- In **homogeneous coordinates**, n -D points are represented by lines in a $(n+1)$ -D space:
 - Additional coordinate w
 - (x,y) represented as (wx,wy,w)
 - Canonical representation:
 $(x,y,1)$
 - Permits additional transformations (e.g., perspective projection) that we do not require in our context

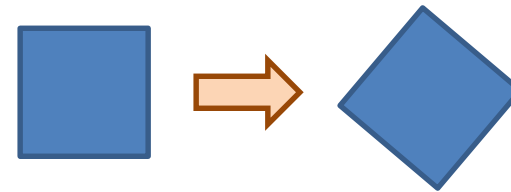


Overview of Affine Transformations

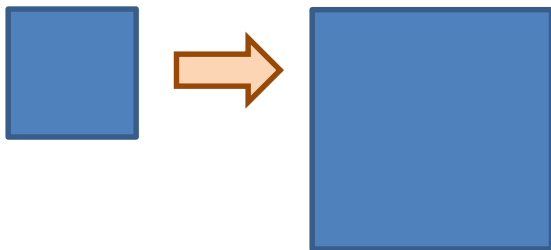
- Affine transformations** can be expressed by multiplying a homogeneous coordinates vector $\mathbf{v}$ to a transformation matrix $\mathbf{M}$: $\mathbf{v}' = \mathbf{M}\mathbf{v}$



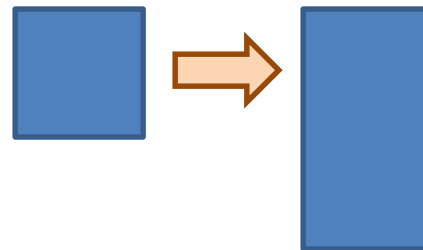
Translation



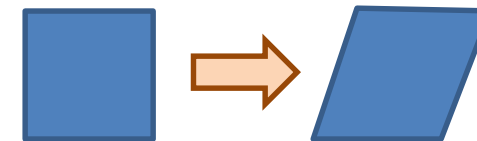
Rotation



Uniform Scaling



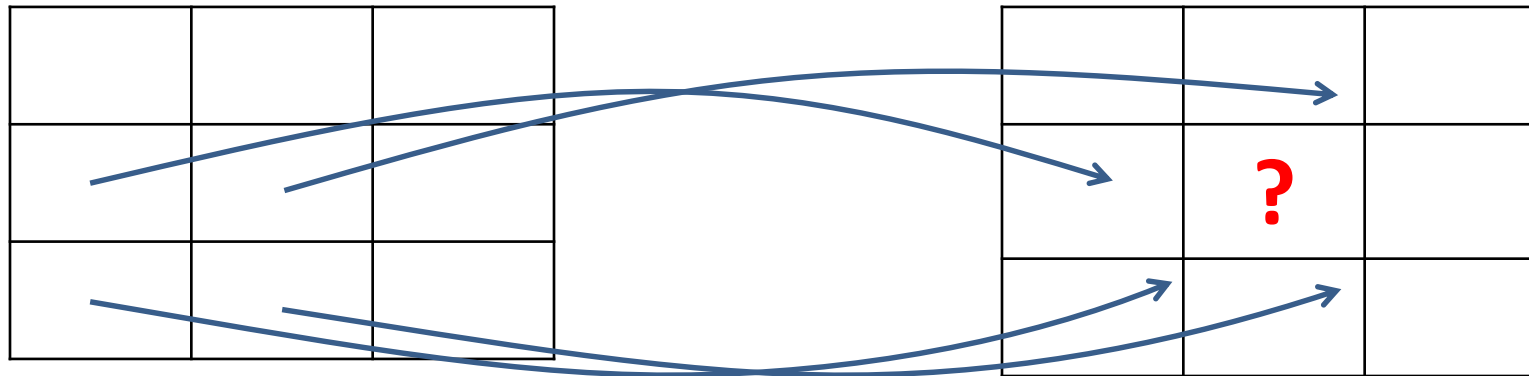
Non-Uniform Scaling



Shear

Image Warps Require Backward Transform

- **Forward transformation** T would leave holes in the final image



- Instead, use **inverse (backwards)** transform T^{-1} to fetch color using interpolation



Storing Voxels in Linear Memory

Computer memory is addressed linearly, so we need to store value at 3D voxel coordinate (i,j,k) at a memory location a

- One option, assuming $n_x \times n_y \times n_z$ voxels:

$$a = k + n_z * (j + n_y * i)$$

Example in Python/numpy:

```
import numpy as np
nx=5; ny=7; nz=4
i=3; j=5; k=2
voxeldata = np.zeros((nx,ny,nz))
voxeldata[i,j,k]=1
a=voxeldata.ravel()
print(a[k+nz*(j+ny*i)])
```

Output: 1.0

Index Coordinates vs. World Coordinates

- The voxel coordinates (i,j,k) from which we compute the index in memory are also called “index coordinates”.
- The 3D “world coordinates” (x,y,z) at which the sample was measured or simulated are computed from (i,j,k) using an origin **p** and three vectors (**e,f,g**) that correspond to a step along the respective array axis:

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \mathbf{p} + i\mathbf{e} + j\mathbf{f} + k\mathbf{g}$$

Index to World as an Affine Matrix

In homogeneous coordinates, the equation

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \mathbf{p} + i\mathbf{e} + j\mathbf{f} + k\mathbf{g}$$

can be re-written as

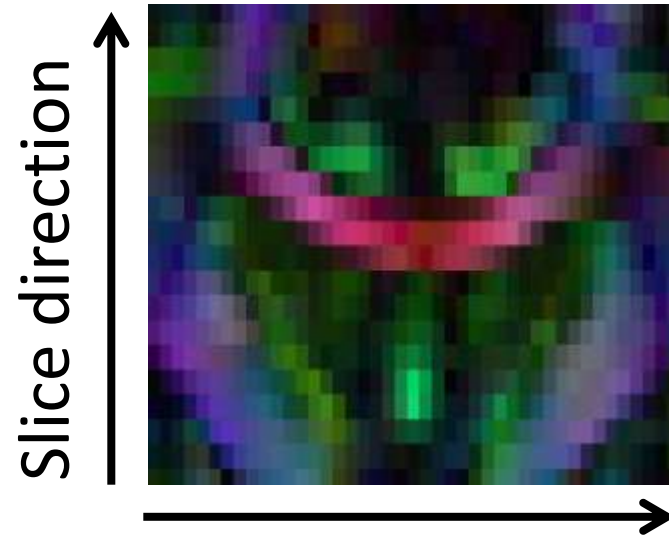
$$\begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} | & | & | & | \\ \mathbf{e} & \mathbf{f} & \mathbf{g} & \mathbf{p} \\ | & | & | & | \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} i \\ j \\ k \\ 1 \end{pmatrix}$$

Voxel Anisotropy

- In most cases, ($\mathbf{e}, \mathbf{f}, \mathbf{g}$) are mutually perpendicular.
- If they also all have the same length, the voxels are called *isotropic*.
- If we make our 2D slices thicker than the in-plane resolution, this leads to *anisotropic voxels*:



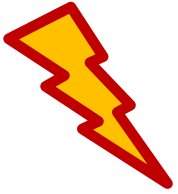
In-plane slice



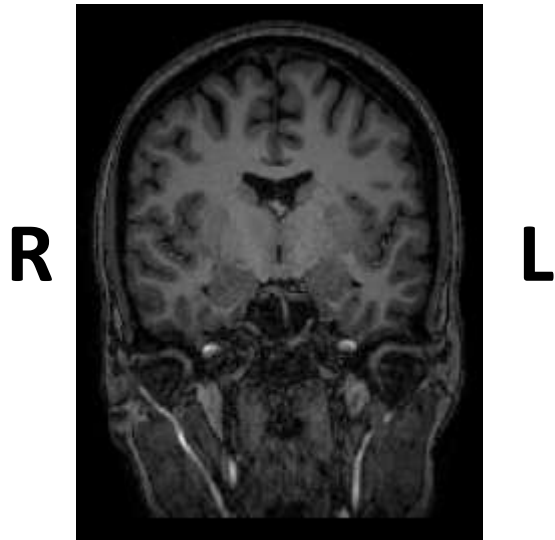
In-plane direction

Handedness of the Coordinate System

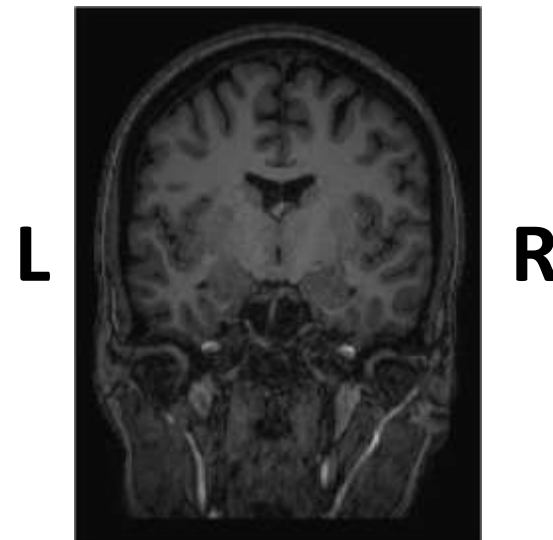
Specifying coefficients of $(\mathbf{e}, \mathbf{f}, \mathbf{g})$ assumes a canonical coordinate system, which can be either right- or left-handed.



Contradictory conventions exist in neuroimaging!



Radiological



Neurological

Orientation in Medical Imaging

Axis directions are often disambiguated by referring to a patient-centered coordinate system:

- *Positive x*: Left or Right?
- *Positive y*: Anterior (front) or Posterior (back)?
- *Positive z*: Superior (top/head) or Inferior (bottom/feet)?

Expressed as **three-letter codes**, e.g.,

- LAS = left-anterior-superior → left-handed
- RAS = right-anterior-superior → right-handed

Summary

We now know about:

- **Affine transformations**, which represent the search space in „linear“ registration
 - computations based on homogeneous coordinates and matrix notation
 - practical application involves backwards warping
- **Index space vs. world space**
 - Issues of voxel anisotropy and handedness

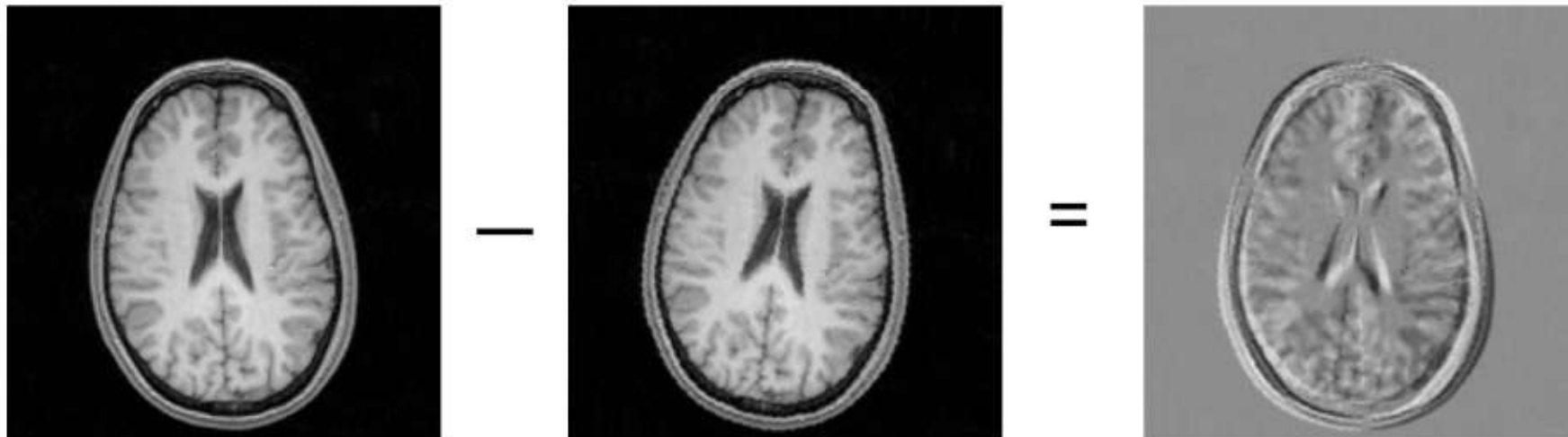
4.3 Cost Functions for Registration

Least-Squares (L2) Cost

- Average squared difference between N pixel values in images X and Y :

$$C^{LS} = \frac{1}{N} \sum_{i=1}^N (x_i - y_i)^2$$

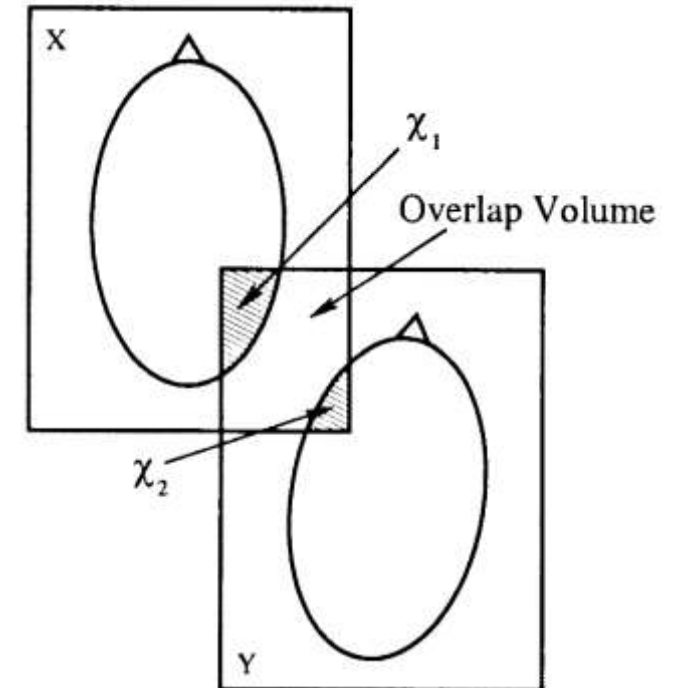
- Intra-modality, but simple and computationally efficient
- To account for global intensity changes, we can add a global scaling term to the optimization („Scaled Least Squares“)



Images from Oxford FMRIB

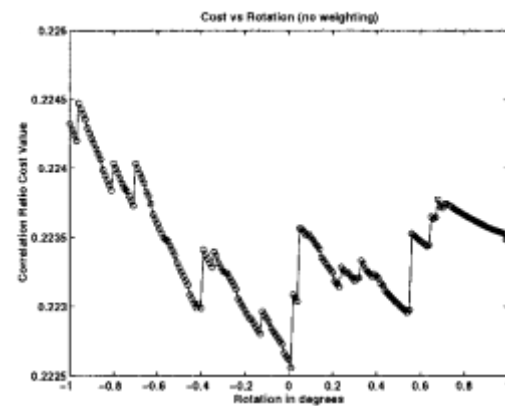
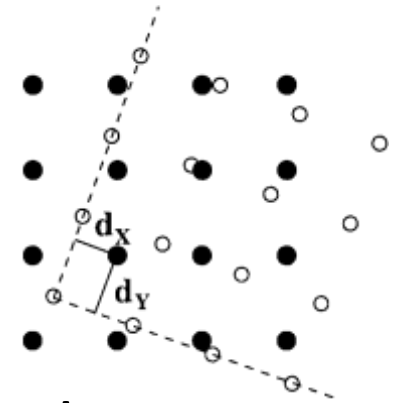
Treating Partial Overlaps

- Fixed and moving images often overlap only partly
 - Due to different image dimensions and/or shifts
- Strategies:
 - Pad with some background value (air)
 - Problematic when objects are only partially within the field of view in the floating image
 - Compute cost function only for overlapping region
 - Size of the domain changes during registration

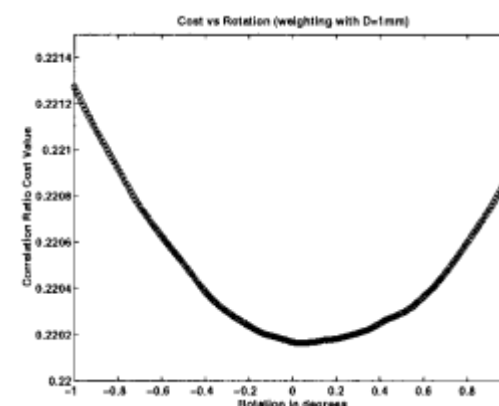


Smoothly Treating Partial Overlaps

- **Problem:** Boundaries cause discontinuities, which cause local minima that complicate optimization
- **Solution:** Compute cost function only for overlapping region. Additionally, *smoothly de-weight* values close to a boundary
 - *Example:* Decrease weight linearly when distance to boundary is less than one voxel



Discontinuous Cost Function



With Smooth De-Weighting

Normalized Cross-Correlation

- Dot product of the two images, normalize by the product of root mean square intensities:

$$S^{NC} = \frac{\sum_{i=1}^N x_i \cdot y_i}{\sqrt{\sum_{i=1}^N x_i^2} \cdot \sqrt{\sum_{i=1}^N y_i^2}}$$

- Also known as “cosine similarity”. Unlike C^{LS} , this must be maximized
- Normalization compensates global re-scaling of intensity
 - Recall that MRI signal has arbitrary units
- Not sufficient for inter-modality registration

Local Cross-Correlation

- Removes local mean (e.g., over a 5x5x5 window) from the image / first, producing image $\bar{I}$:

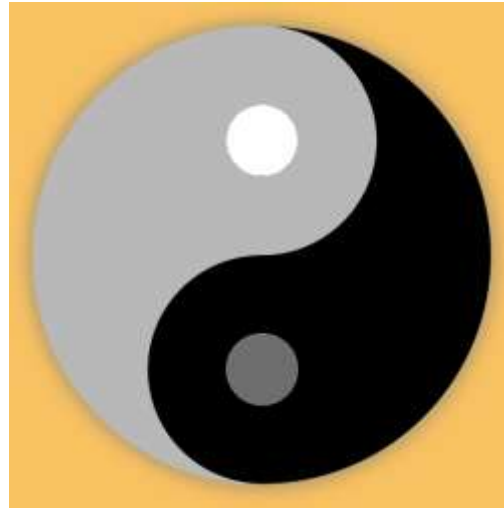
$$S^{LNC} = \frac{\sum_{i=1}^N \bar{x}_i \cdot \bar{y}_i}{\sqrt{\sum_{i=1}^N \bar{x}_i^2} \cdot \sqrt{\sum_{i=1}^N \bar{y}_i^2}}$$

- Allows us to deal with bias fields in MRI

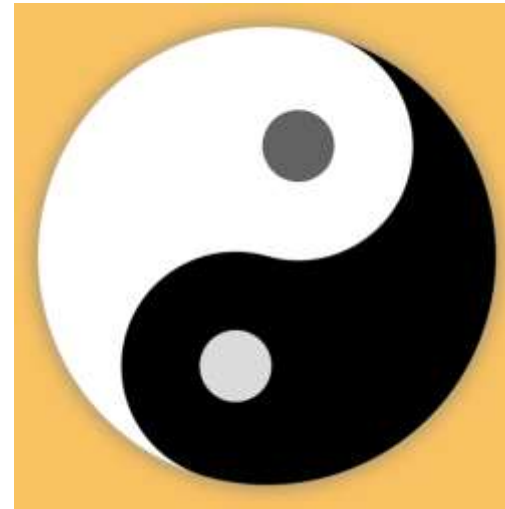


Idea: Variance of Intensity Ratios

- **Idea:** In a given image, voxels with the same intensity should contain the same object
- **Conclusion:** If we only consider voxels with similar intensity in X, the **variance of intensity** in corresponding voxels in Y should be small



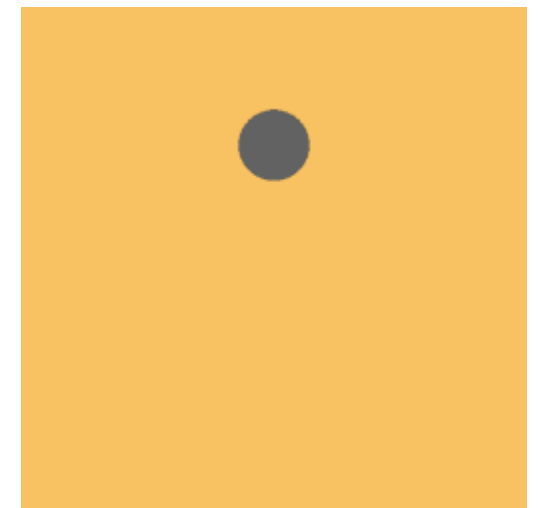
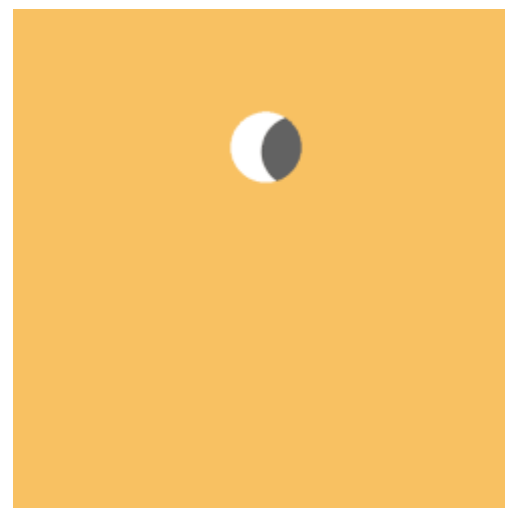
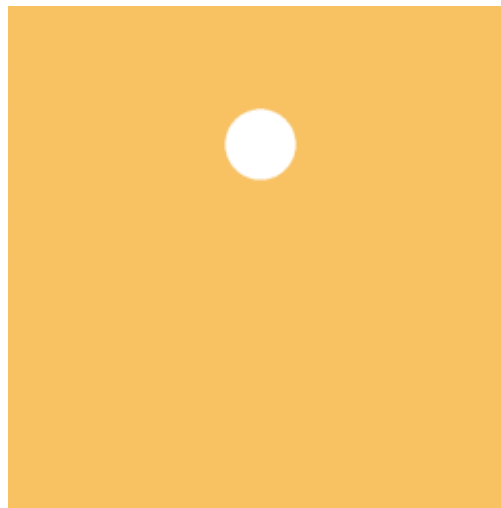
X



Y (misaligned)



Y (aligned)



Equation: Variance of Intensity Ratios

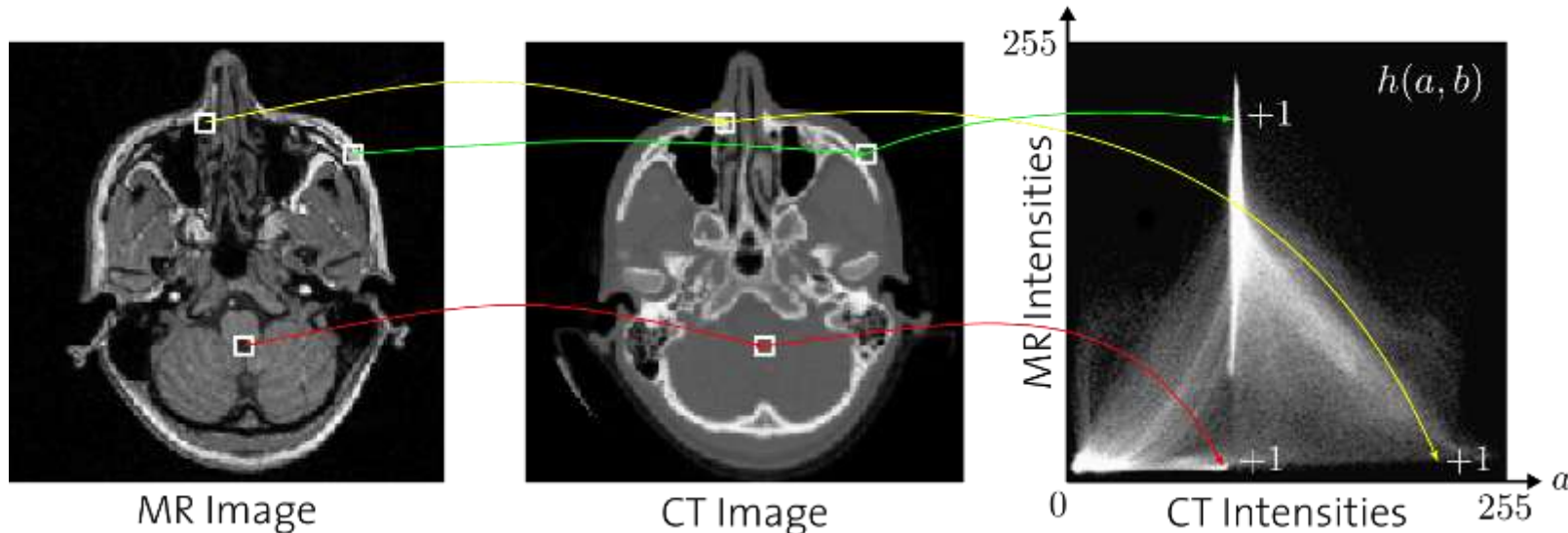
- **Definition:** Partition intensities in the reference image X into K bins (disjoint intervals). For bin k , the set Y_k contains the corresponding intensities from the same locations in the *other* (moving) image Y . Then, compute an average normalized standard deviation over all Y_k :

$$C^{VIR} = \sum_{k=1}^K \frac{|Y_k|}{N} \frac{\sigma(Y_k)}{\mu(Y_k)}$$

- First widely used intensity-based *inter-modality* cost function, proposed by Woods in 1993

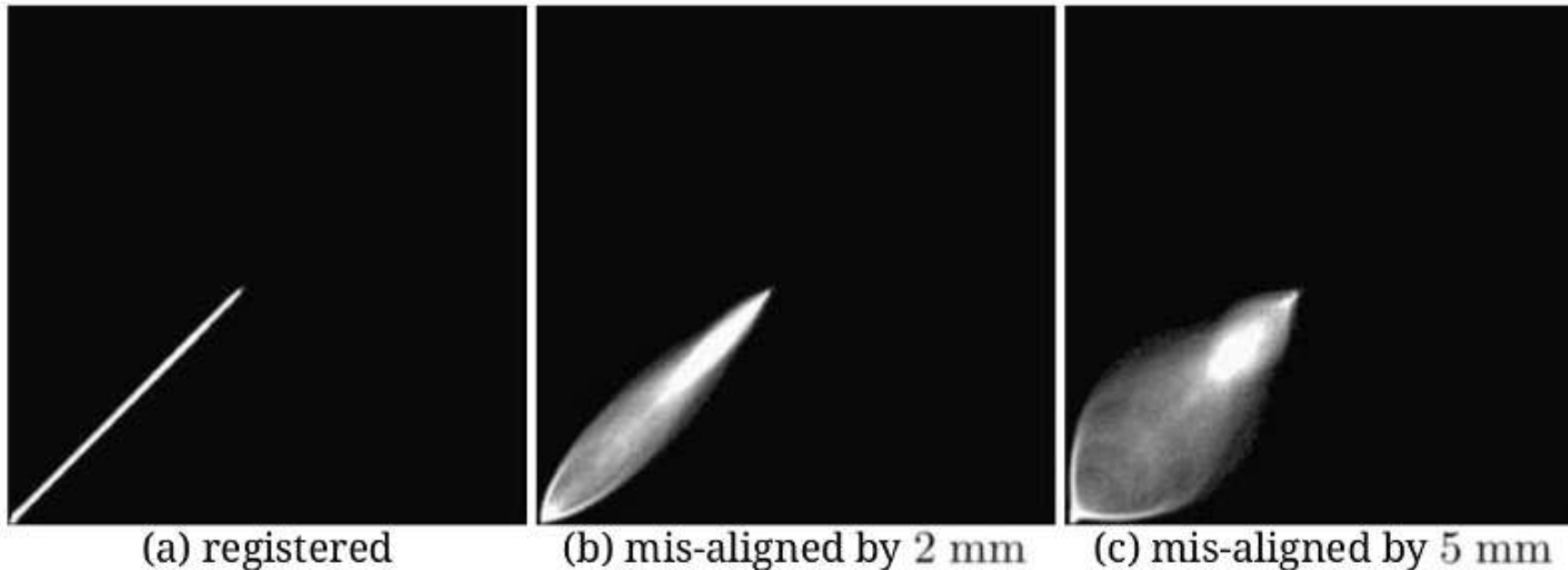
Joint Distributions

- C^{VIR} is one example of a family of inter-modality cost functions that rely on **joint distributions** of the two images
- Joint distribution represented by 2D histogram
 - One image per axis
 - Value at (x,y) answers the question: How frequently does intensity x in image X coincide with intensity y in image Y ?



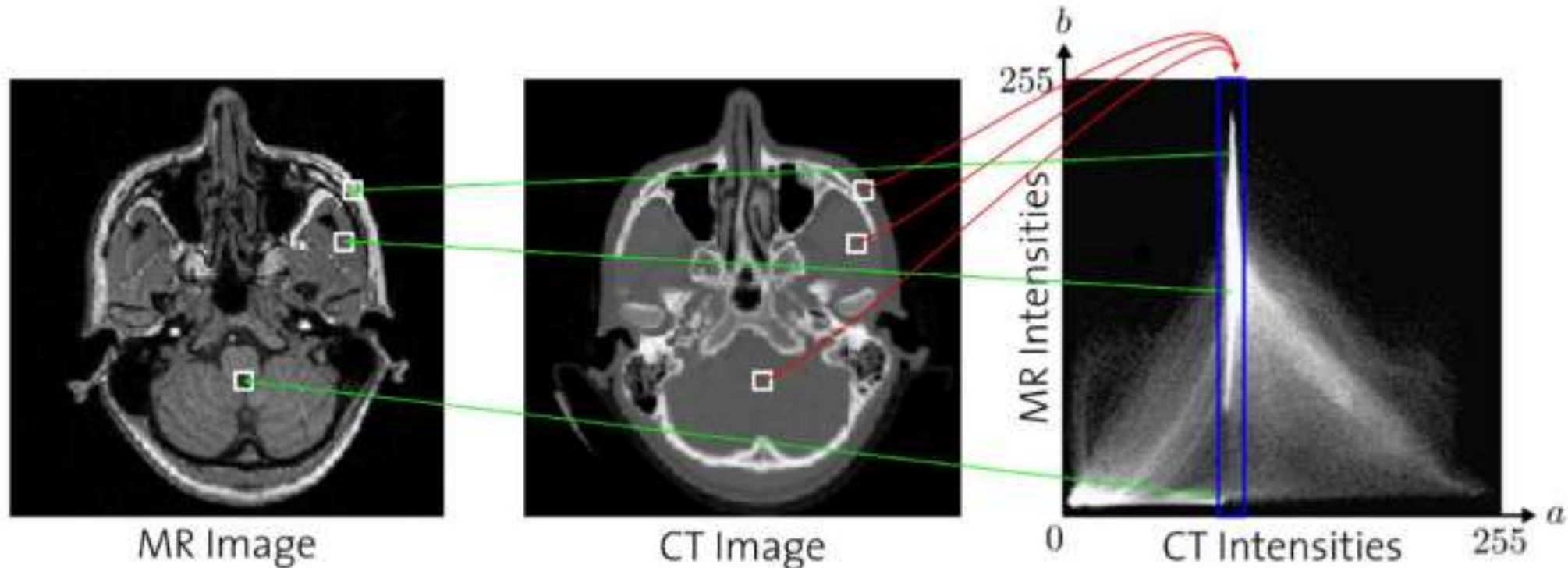
2D Histograms for Registration

- Visually, 2D histograms of aligned images tend to look „sharp“ or „structured“
 - Structure disperses with misalignment
 - S^{NC} and C^{VIR} quantify specific structures



Intensity Mapping May Not Be One-to-One

- Some imaging modalities can differentiate tissue (sub-)types that look uniform with another imaging modality
 - Mapping of intensities may not be one-to-one
 - Violates the assumption of $\mathcal{C}^{VIR}$



Large Scalings

- It is desirable that a cost function should attain its minimum *only* when comparing two identical images
- Consider what happens when we scale (enlarge) the image Y so strongly that a single voxel covers all of X :
 - For all k , $\sigma(Y_k) = 0$
 - C^{VIR} is minimized for this unreasonable transformation
 - Optimizing C^{VIR} might not produce the correct result!

Information in Images

- **Information** in the sense of Shannon
 - Quantifies “surprise” from revealing pixel values
 - No information if all pixels have the same intensity
 - **Entropy** H quantifies the amount of uncertainty in a discrete distribution with probabilities $p_1, p_2, \dots, p_K$:

$$H = - \sum_{i=1}^K p_i \log_2 p_i$$

- If p_i are relative frequencies from the histogram of image X , $H(X)$ is the information content of X
- *Note:* This ignores the spatial structure!

Joint Entropy

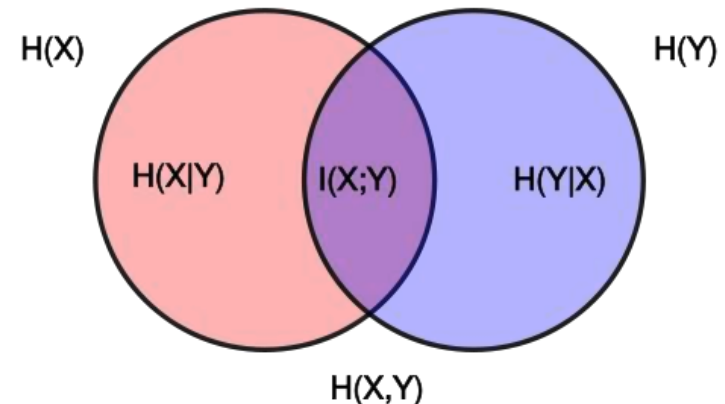
- **Joint entropy** $H(X,Y)$ is the entropy of the joint distribution (2D histogram) of X and Y :

$$H(X, Y) = - \sum_{i=1}^{K_X} \sum_{j=1}^{K_Y} p_{ij} \log_2 p_{ij}$$

- *Intuition*: Registration makes it easier to predict values in Y from values in X , thus reduces information in the joint distribution
- Captures more general structure in the joint distribution than S^{NC} and C^{VIR}
- Still not optimal: Joint entropy is minimized by excessive scaling

Mutual Information

- Degree of overlap between images X and Y (and therefore the entropies $H(X)$ and $H(Y)$ in the overlapping region) varies during registration
 - This causes the problem with excessive scaling:
The entropy of the much enlarged image is $H(Y)=0$
 - This is better accounted for by **Mutual Information**
[Viola/Wells 1995, Collignon et al. 1995]
$$I(X, Y) = H(X) + H(Y) - H(X, Y)$$
 - Registration should *maximize* mutual information
 - Quantifies the *dependence* of two variables (i.e., how much information one provides about the other)

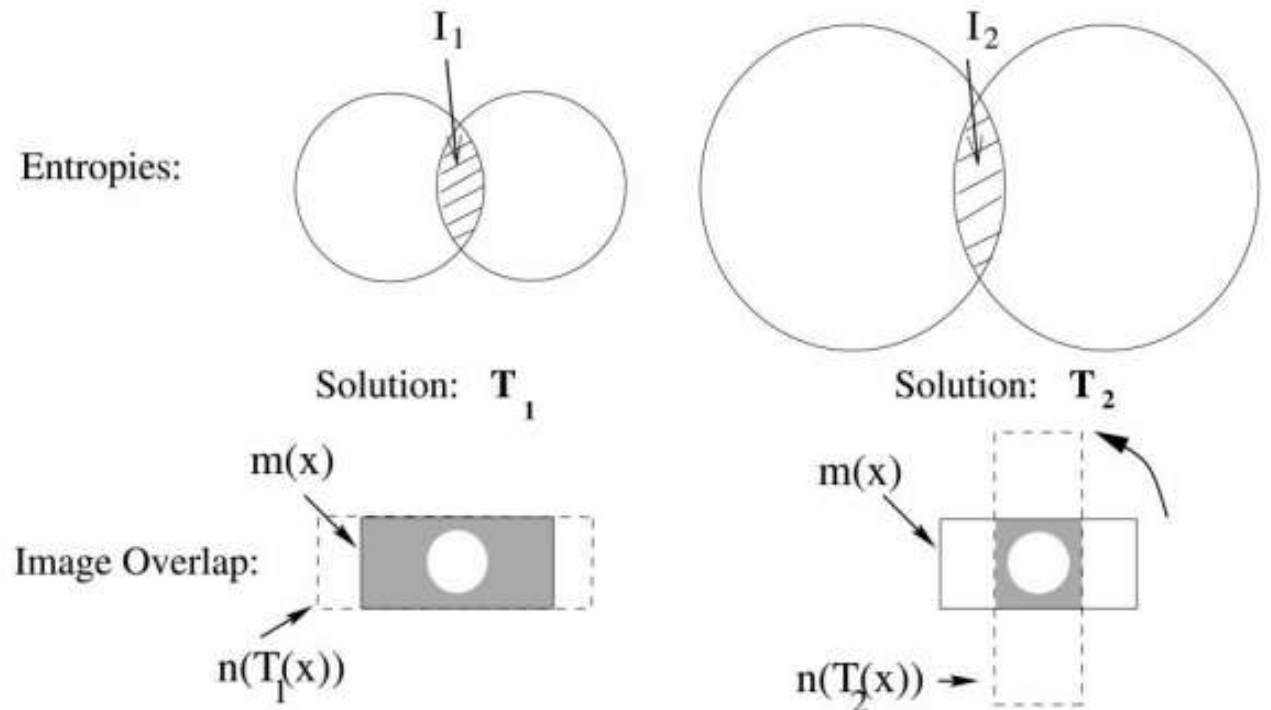


Normalized Mutual Information

- **Normalized Mutual Information** [Studholme et al. 1999] has proven to work even better in cases of varying image overlaps:

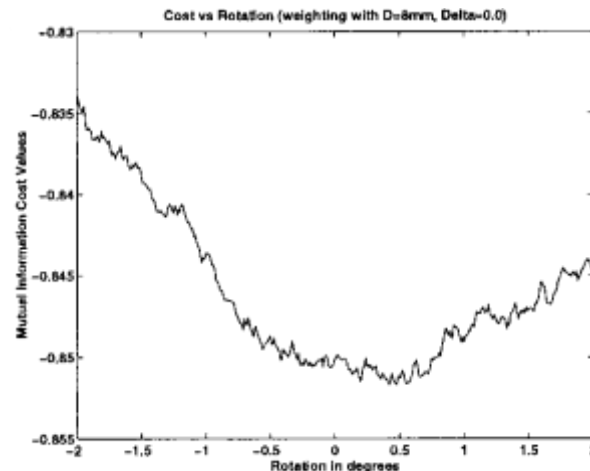
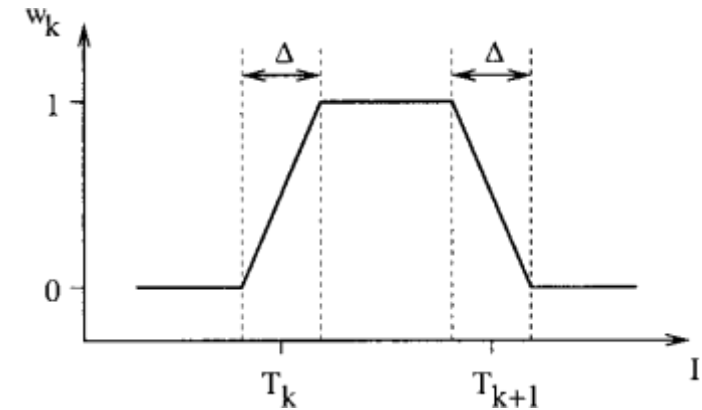
$$NMI(X, Y) = \frac{H(X) + H(Y)}{H(X, Y)}$$

- Should be maximized,
like $I(X, Y)$

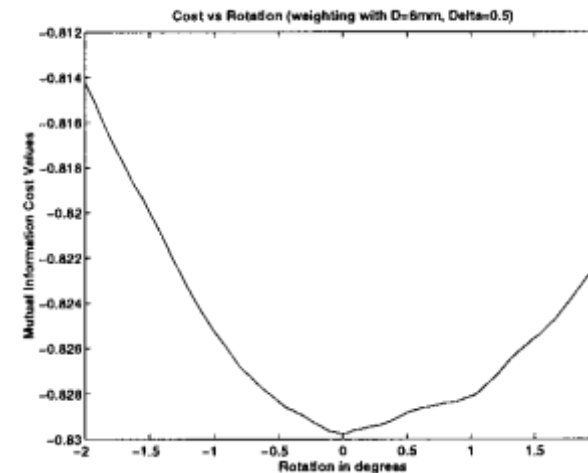


Fuzzy Binning

- Creating 2D histograms via binning is discontinuous
- Causes local minima
- **Solution:** Create bins with a soft (fuzzy) boundary
 - Combine with boundary de-weighting



Discontinuous Mutual Information



With Fuzzy Binning

Summary: Registration Cost Functions

- Optimization-based registration methods in neuroimaging use **intensity-based cost functions**
 - Computed over overlapping volume
- **Intra-modality** cost functions
 - Compare image intensity directly
 - Least Squares and (Local) Cross-Correlation
- **Inter-modality** cost functions
 - Based on joint (2D) histogram
 - (Normalized) Mutual Information
- Soft weights at image and bin boundaries help avoid **local minima**

4.4 Optimization For Registration

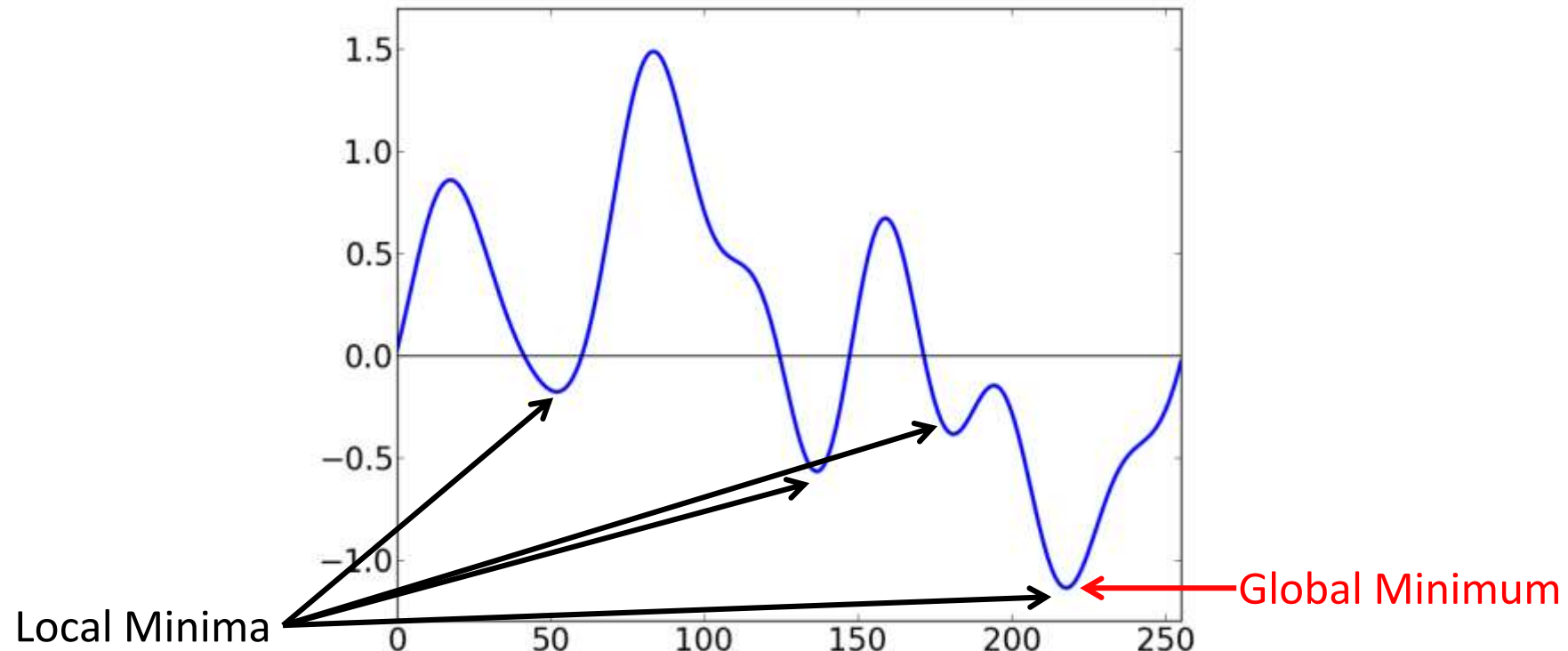
Optimization Problem in Registration

- Given
 - Reference image $\mathbf{y}$
 - Moving image $\mathbf{x}$
 - Cost function C
 - Space S_T of allowable transformations, parameterized by $\mathbf{w}$
- The transformation T^* that achieves registration is found by the following optimization:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} C(\mathbf{y}, T(\mathbf{x}|\mathbf{w}))$$

Goal in Optimization

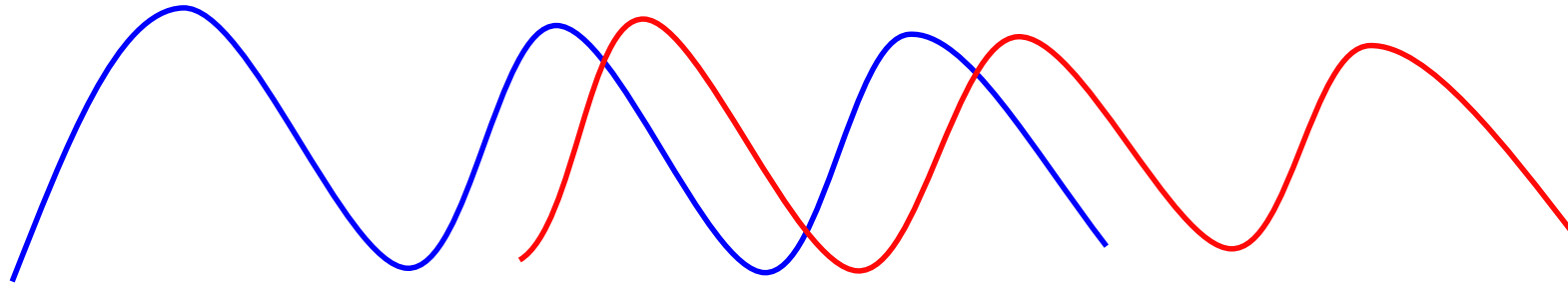
- Methods for optimization search a minimum of a given function $f(x)$
 - Maxima can be found by considering $-f(x)$



Local vs. Global Optimization

- **Local Optimization:**

- Given an initial guess x , find a nearby local minimum
- Rarely sufficient for registration:



- **Global Optimization:**

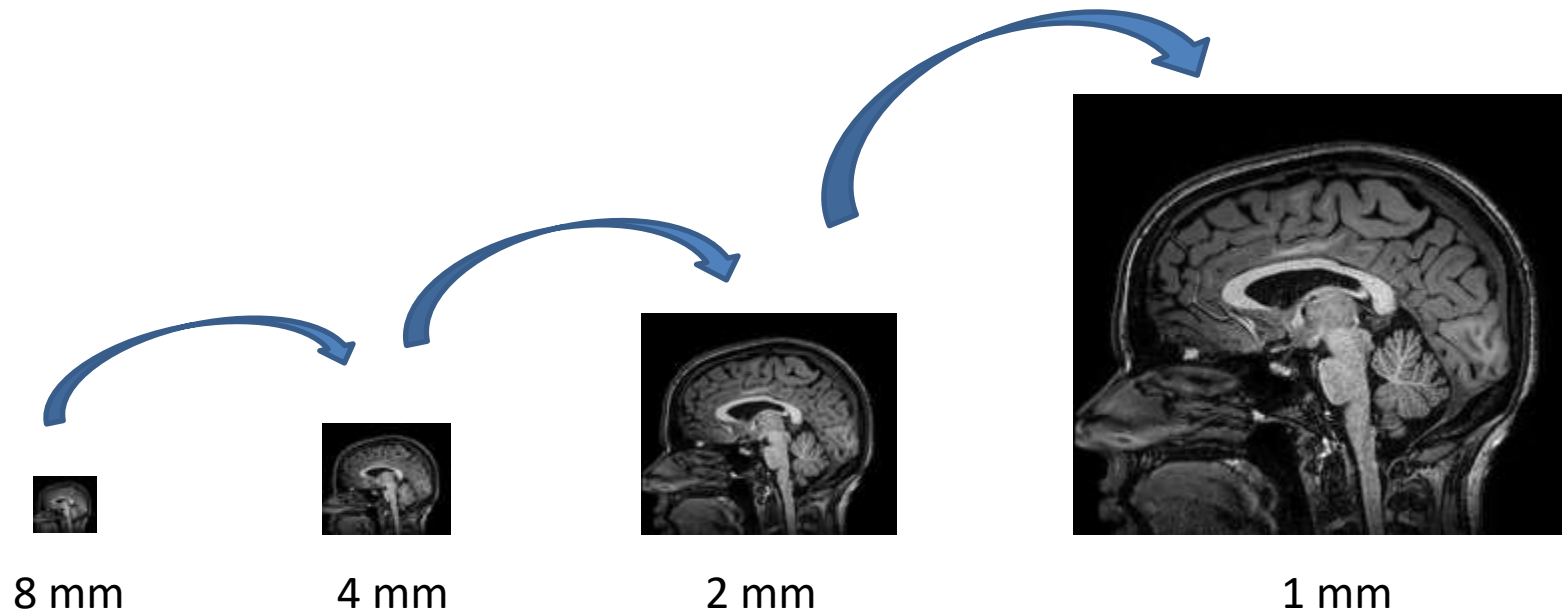
- No computationally efficient method guarantees to find the global optimum of unconstrained functions $f(x)$
- Heuristics can help us find a useful optimum in most practical cases
- Manual intervention might still be needed

Grid Search

- **Idea:** To avoid dependency on initial guess, we could systematically try different initializations
 - *Example:* 10 samples within a reasonable range
- **Problem:** Combinatorial explosion in high dimension
 - *Example:* Full affine: 10^{12} combinations to try
 - If we can evaluate the cost function within 1/10th of a second, this will take over 3000 years!
- Useful if dimensionality is small and cost function evaluation is fast

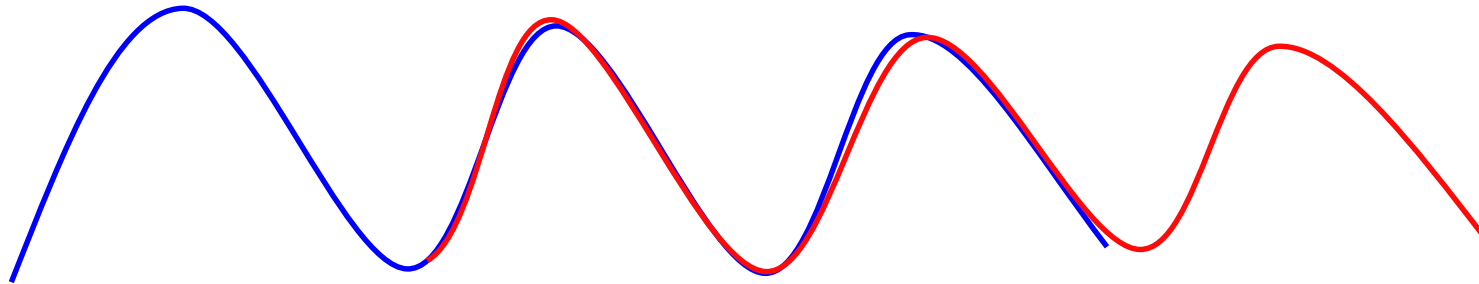
Multi-Scale Optimization

- **Idea:** Achieve a fast rough alignment on a coarser scale
 - Cost function very fast to evaluate, few steps needed
 - Also hope to avoid some local minima this way
- **Assumption:** Desired optimum at fine scale is within basin of attraction of optimum at coarser scale

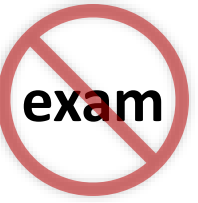


Perturbation and Re-Start

- Randomly perturbing the result and re-starting optimization can lift us out of local optima
 - *But:* It could also push us into another, worse local optimum, so compare to previous optimum!



Real-World Example: FSL's FLIRT

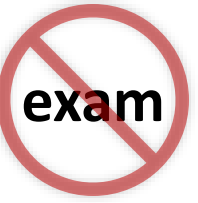


- **FLIRT** (FMRIB's Linear Image Registration Tool) is widely used for affine registration of brain images
 - Part of FSL, <http://fsl.fmrib.ox.ac.uk/>
 - >8500+14300 citations (Nov. 2025) according to google scholar

General strategies of FLIRT's approach:

- **Domain-specific pragmatic / empirical choices**
- **Gradually increase complexity**
 - Multi-scale approach
 - Initial coarse grid search, refine later
 - Optimize with fewer degrees of freedom first

Initial Grid Search in FLIRT

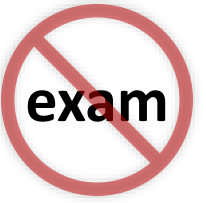


Observation: Finding correct rotation is most difficult part in practice

Initial Strategy: Multi-stage search at 8mm

1. Sample a coarse grid of rotations
 - 60° increments around three axes, $6^3=216$ rotations
 - Optimize translation and scale for each of them
2. Sample a finer grid of rotations
 - 18° increments, $20^3=8000$ rotations
 - Evaluate cost function once, for median scale and interpolated translation from stage 1
3. Perform full local optimization for each local minimum found at stage 2

Registration Refinement in FLIRT



- The best three local optima from 8 mm resolution are further refined at 4 mm
 - Each result is perturbed 10 times with respect to translation, rotation, and scale
- The single best result from 4 mm is further refined at 2 mm
 - After similarity transform, anisotropic scaling and shear are added to candidate transformations
- A 12 DOF optimization run at 1 mm resolution yields the final result

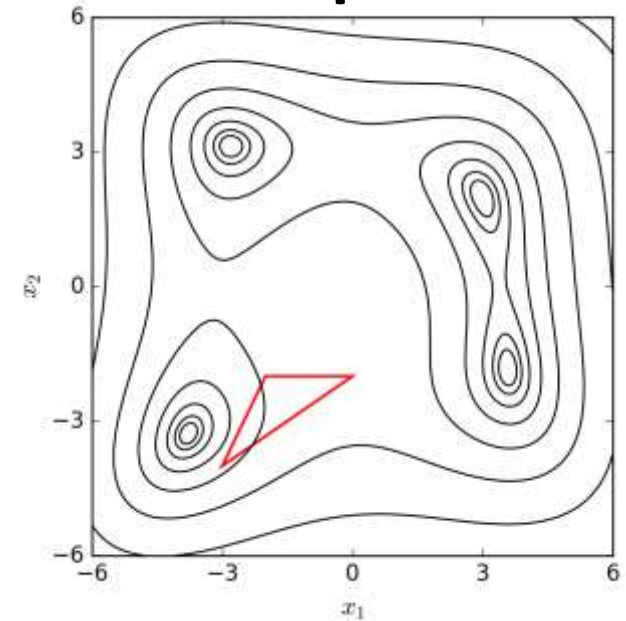
Derivative-based vs. Derivative-Free

- **Derivative-free** optimization methods treat $f(x)$ as a „black box“. We are only able to evaluate it at different points x
 - Only assumption: $f(x)$ is continuous
 - Boundary and binning effects can make $f(x)$ non-differentiable
 - Will be covered in Section 4.5
- **Derivative-based** methods additionally make use of first or higher-order derivatives
 - Assume that $f(x)$ is smooth
 - *Special case*: Cost function is a sum of squared error terms
 - Will be covered in Section 4.6

4.5 Derivative-Free Optimization

Basic Idea: Downhill Simplex Method

- The **downhill simplex (Nelder-Mead) method** is a simple and robust approach for the derivative-free minimization of functions in d variables
- It is based on an initial position $\mathbf{p}_0$ and d points $\mathbf{p}_i$ around it that span all possible search directions
 - The convex hull of $d + 1$ points in d -dimensional space is a **simplex**
 - line segment in 1D, triangle in 2D, tetrahedron in 3D, etc.
 - Test point with lowest function value is our current best estimate
 - Main strategy is to iteratively improve the worst point, or contract the simplex, until some convergence criterion is met (“amoeba method”)



Initializing the Simplex

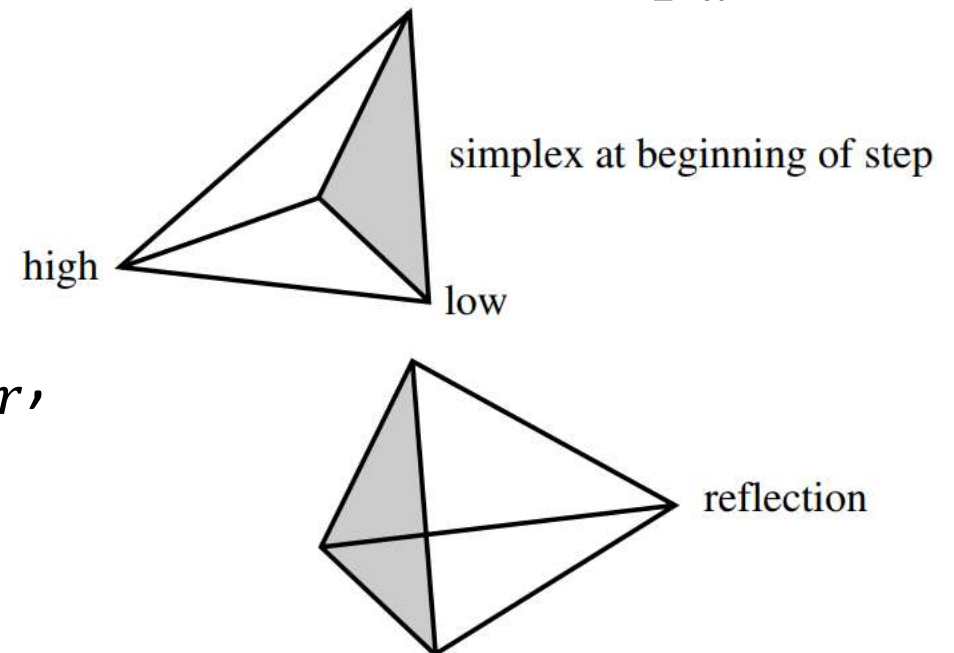
- The simplex needs to be **non-degenerate** so that the algorithm can search the whole d -dimensional parameter space
 - When initialized accordingly, iterations maintain this property
- The **distance** between points corresponds to the scale of the currently considered neighborhood in parameter space
- A natural strategy for initialization is via an initial guess $\mathbf{p}_0$ and a characteristic length scale Δ_i of each parameter:

$$\mathbf{p}_i = \mathbf{p}_0 + \Delta_i \mathbf{e}_i$$

- Initial step sizes Δ_i need to account for the expected magnitudes of transformations, as well as chosen units (e.g., shifts in m or mm, rotations in degree or radian)

1. Reflection

- We first attempt to move the highest point to a better position while maintaining the volume of the simplex
 - Sort points $\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_d$ so that the corresponding functions values are in increasing order $f_0 \leq f_1 \leq \dots \leq f_d$
 - Obtain a new test point $\mathbf{p}_r$ by reflecting the highest point $\mathbf{p}_d$ through the opposite face:
$$\bar{\mathbf{p}} := \frac{1}{d} \sum_{i=0}^{d-1} \mathbf{p}_i$$
$$\mathbf{p}_r := \mathbf{p}_d + 2(\bar{\mathbf{p}} - \mathbf{p}_d)$$
 - If $f_0 \leq f_r < f_{d-1}$, replace $\mathbf{p}_d$ with $\mathbf{p}_r$, renumber points, and continue with the next iteration

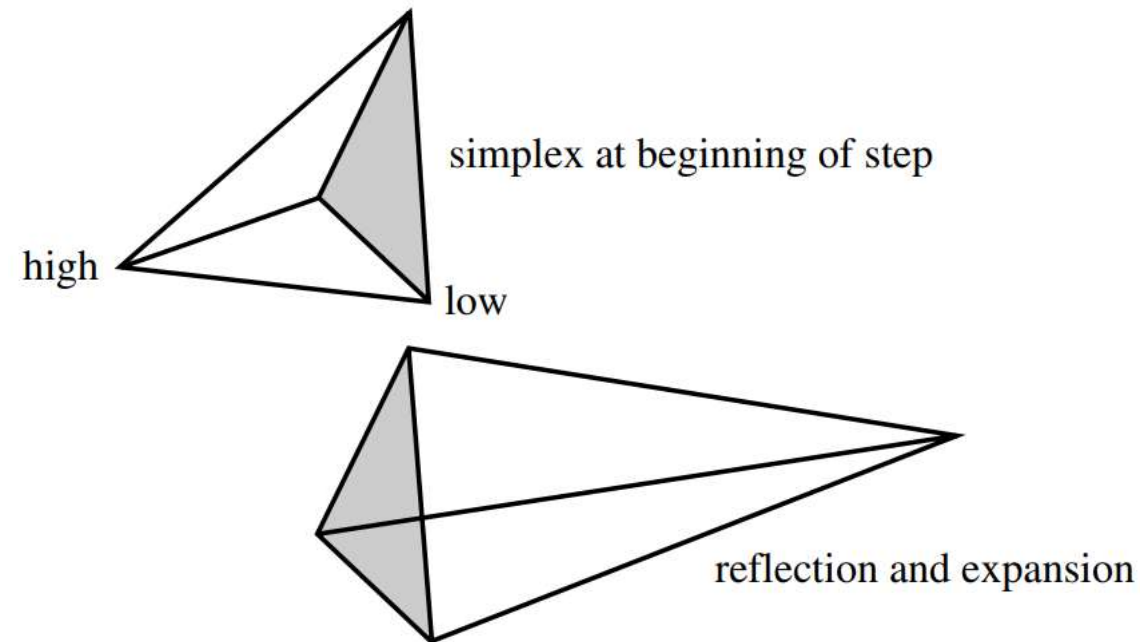


2. Reflection and Expansion

- If the reflected point $\mathbf{p}_r$ is the new low point ($f_r < f_0$), the reflection corresponded to a promising search direction
 - Attempt to expand the simplex by extrapolating in the same direction:

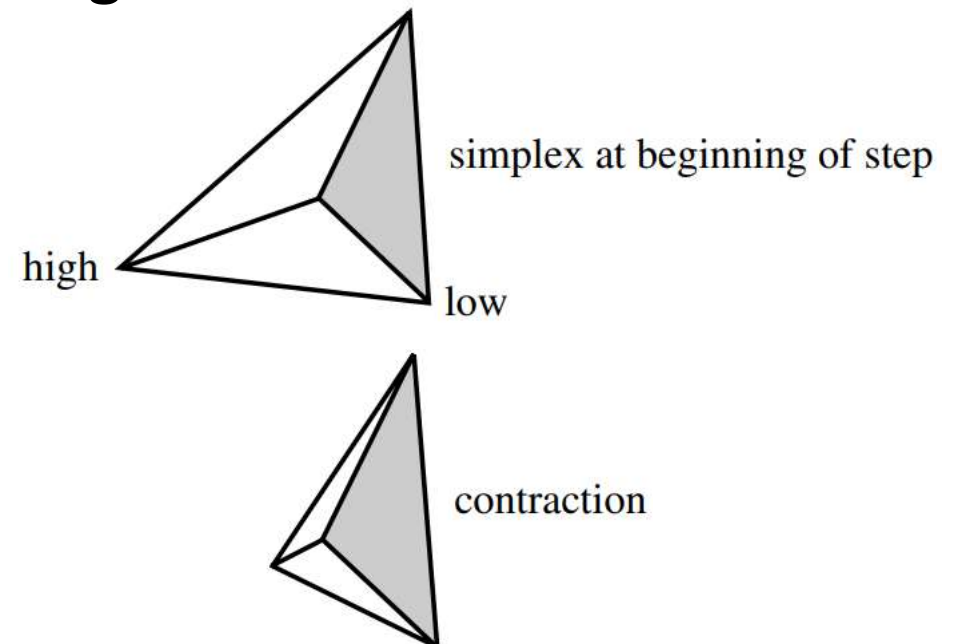
$$\mathbf{p}_e := \mathbf{p}_d + 3(\bar{\mathbf{p}} - \mathbf{p}_d)$$

- If $f_e < f_r$, replace $\mathbf{p}_d$ with $\mathbf{p}_e$,
else replace $\mathbf{p}_d$ with $\mathbf{p}_r$
- Renumber points, continue with the next iteration



3. Contraction

- If the reflected point $\mathbf{p}_r$ is worse than the second highest ($f_r \geq f_{d-1}$), Step 1 would reflect it back in the next iteration
 - Attempt to contract the simplex by moving halfway towards the opposite face: $\mathbf{p}_c := 0.5(\bar{\mathbf{p}} + \mathbf{p}_d)$
 - Makes future steps in the corresponding direction more careful
 - If $f_c < f_d$, replace $\mathbf{p}_d$ with $\mathbf{p}_c$, renumber points, continue with the next iteration

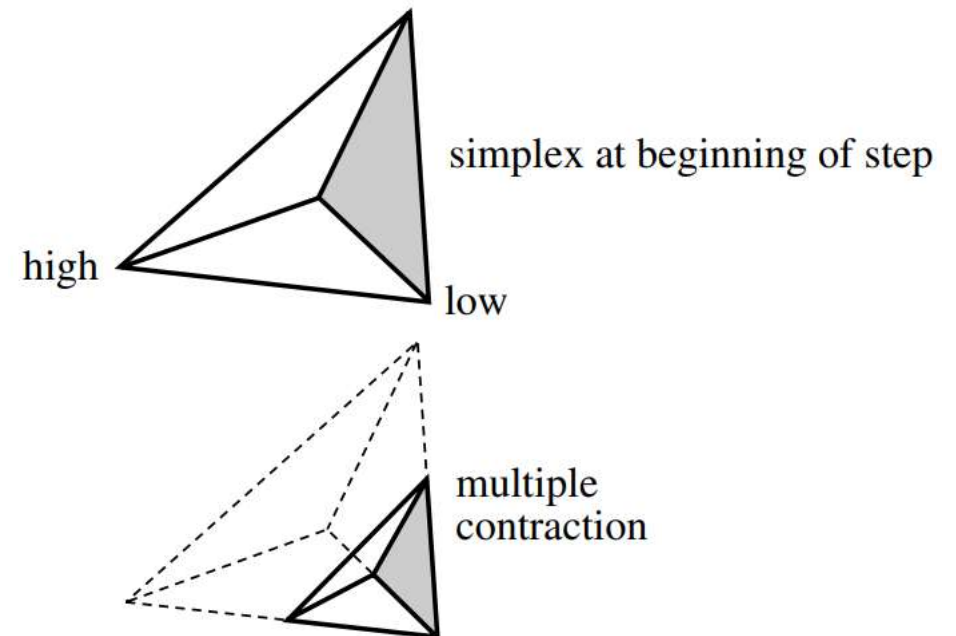


4. Multiple Contraction

- If we could not improve the highest point, even by contracting it, perform a **multiple contraction** of all remaining points around the lowest one:

$$\mathbf{p}'_i := 0.5(\mathbf{p}_i + \mathbf{p}_0)$$

- Leads to more careful future steps in general
- Evaluate f at all new positions and renumber points, continue with the next iteration

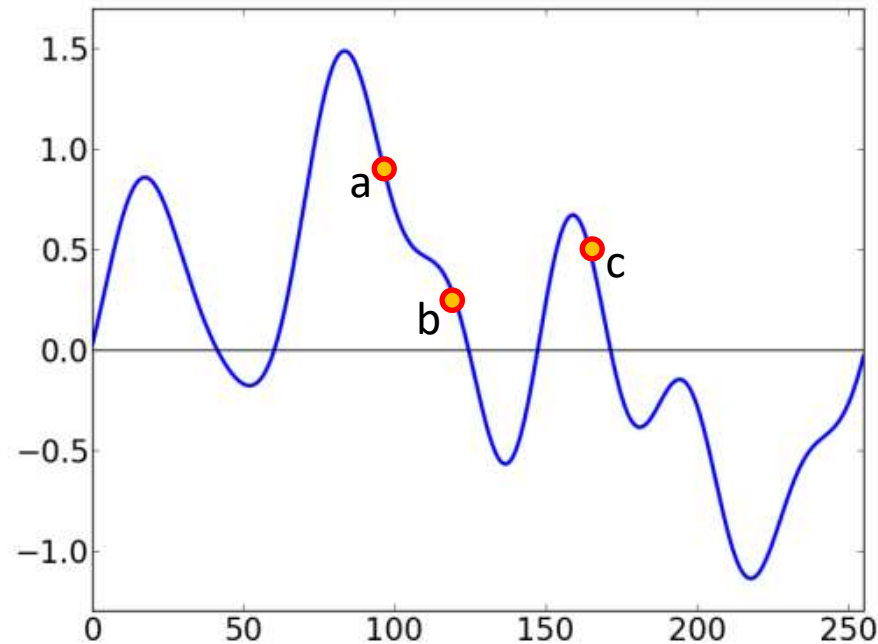


Declaring Convergence

- Typical criteria for **stopping** the downhill simplex method are:
 - the absolute difference $(f_d - f_0)$ or fractional difference $\frac{2(f_d - f_0)}{|f_d| + |f_0| + \epsilon}$ of function values at our test points is below some threshold
 - the maximum distance $\|\mathbf{p}_0 - \mathbf{p}_i\|$ (over $i > 0$) is below some threshold, or all per-axis distances are below axis-specific thresholds
 - some maximum number of iterations is reached
 - some maximum number of function evaluations is reached
- These criteria do *not* guarantee proximity to a local minimum
 - It usually makes sense to re-initialize and try to improve further

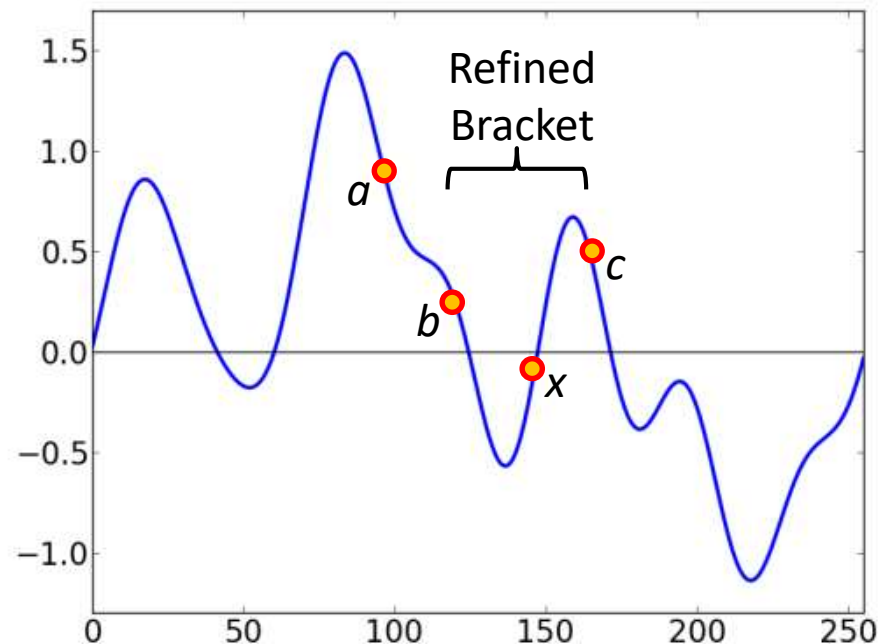
Alternative Idea: Bracketing a Minimum in 1D

- **Observation:** If $f(x)$ is continuous and we have found three values $a < b < c$ such that $f(a) > f(b)$ and $f(c) > f(b)$, at least one local minimum must exist between a and c
 - The three points „bracket“ at least one minimum



Iteratively Refining the Bracket

- **Basic idea:** Given a bracket (a,b,c) , compute $f(x)$ for a new point $x \neq b$ such that $a < x$ and $x < c$
 - Either (a,b,x) or (b,x,c) will meet the bracketing condition. Pick that (now smaller) bracket and subdivide it further, until $|c-a| < \theta$

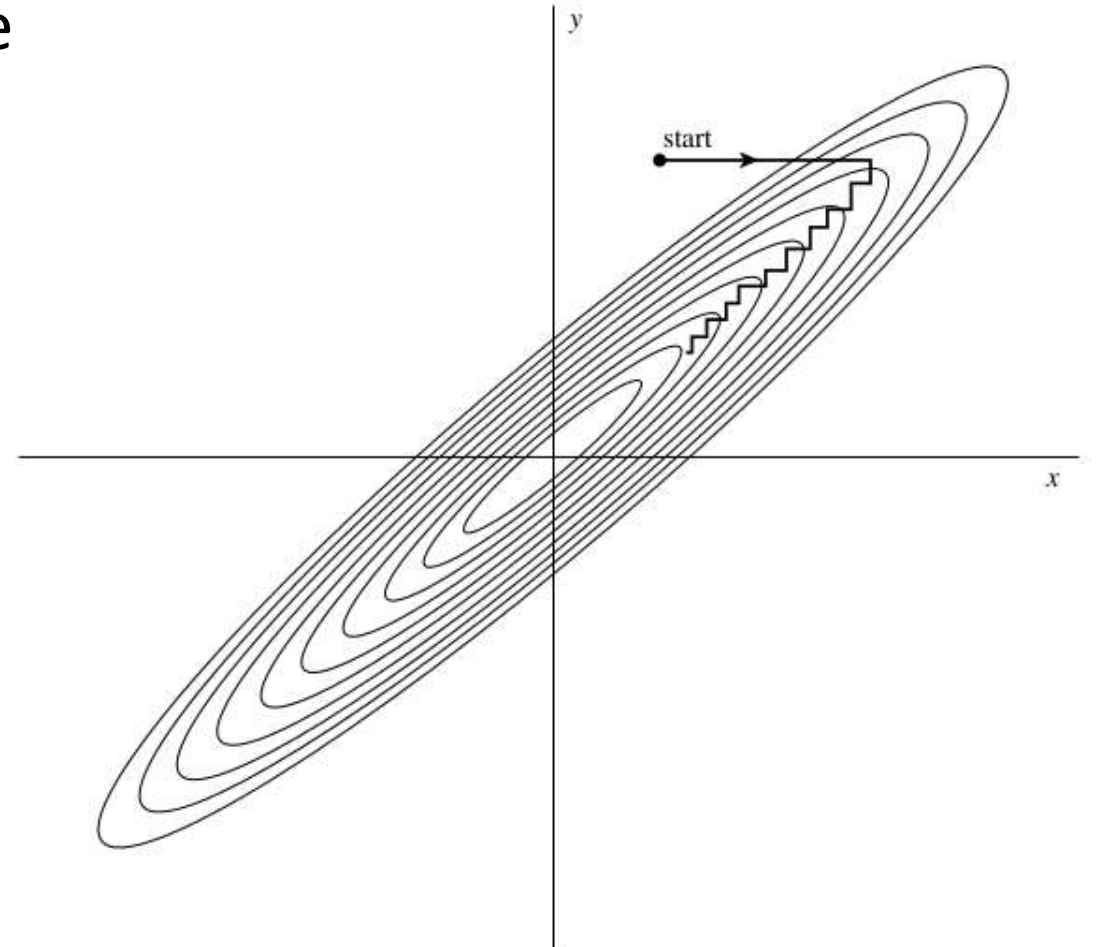


Optimization in High Dimension

- Linear registration works in a 6- (rigid) to 12-dimensional (full affine) space
 - No high-dimensional analog of bracketing exists
 - Instead: Perform 1D optimizations along a set of search directions
 - **Example:** N basis vectors of N -D space
 - Repeatedly cycle over N search directions until convergence
 - We still need similar convergence criteria as in the downhill simplex method
 - Still no guaranteed maximum distance towards the closest local minimum

Challenges for Line Methods

- **Problem:** Alternating 1D optimizations can be very slow for narrow valleys that are not aligned with any search direction
 - Forces the method to take a large number of small steps



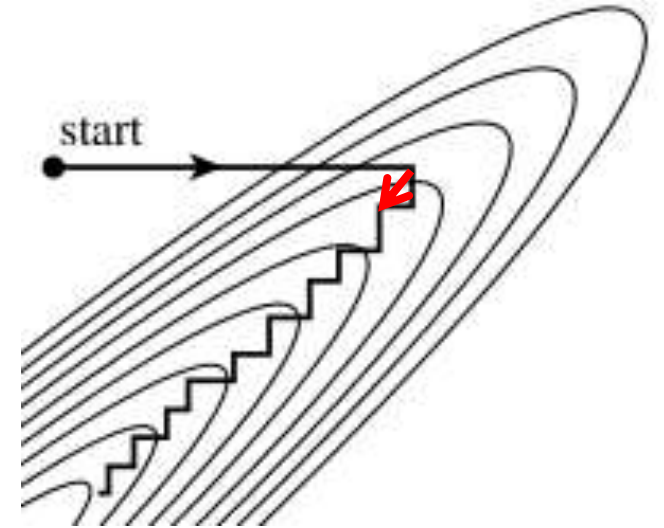
Powell's Method

- **Basic idea:**

- Starting at $\mathbf{p}_0$, perform d line optimizations
- Each 1D optimization produces a new point $\mathbf{p}_i$
- After d 1D optimizations, use $\mathbf{p}_i - \mathbf{p}_{i-d}$ as a new search direction
 - Direction of overall progress
 - Hopefully, will „point along valley“

- **Variants:**

- Append new direction, drop first direction
 - Set of directions can degenerate (become dependent)
- Replace direction of largest decrease
 - Aims to avoid linear dependence
- Orthogonalize search directions



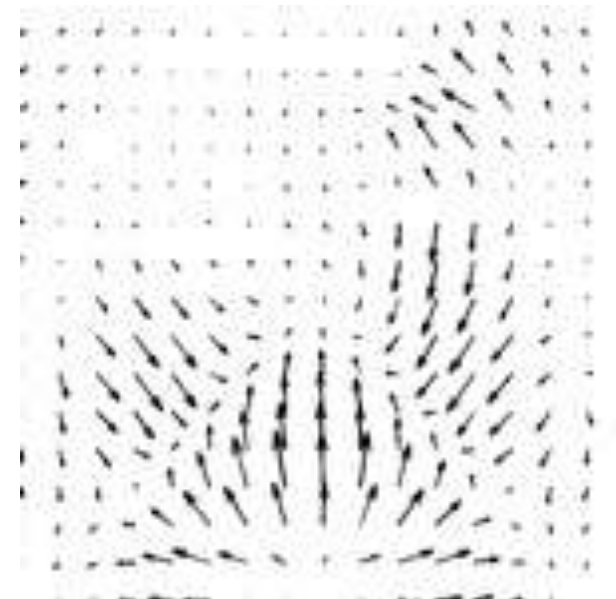
Summary: Derivative-Free Optimization

- **Bracketing** is a “failsafe” approach in 1D, but no direct generalization for multiple dimensions
 - Powell’s method attempts to find a suitable direction for line search
- **Downhill simplex search** is a simple and robust (although not very efficient) approach for derivative-free minimization
 - Maintains $d + 1$ test points that form a d -dimensional simplex
 - Iteratively attempts to improve the worst (highest) point
 - Contracts the simplex if needed to make progress, expands it when it comes across a promising search direction
 - “Convergence” does not guarantee that a local minimum has been found, the method may “get stuck”
 - re-initialization can lead to further progress

4.6 Nonlinear Registration

Nonlinear Transformations

- **Rigid transformations** are used to coregister images of the same brain
 - Affine transformations can compensate some image distortions and serves as an initial step towards between-subject registration
- Matching different brains requires much greater flexibility:
Nonlinear transformations
 - *Basic idea:*
Displacement vector field („warp field“)

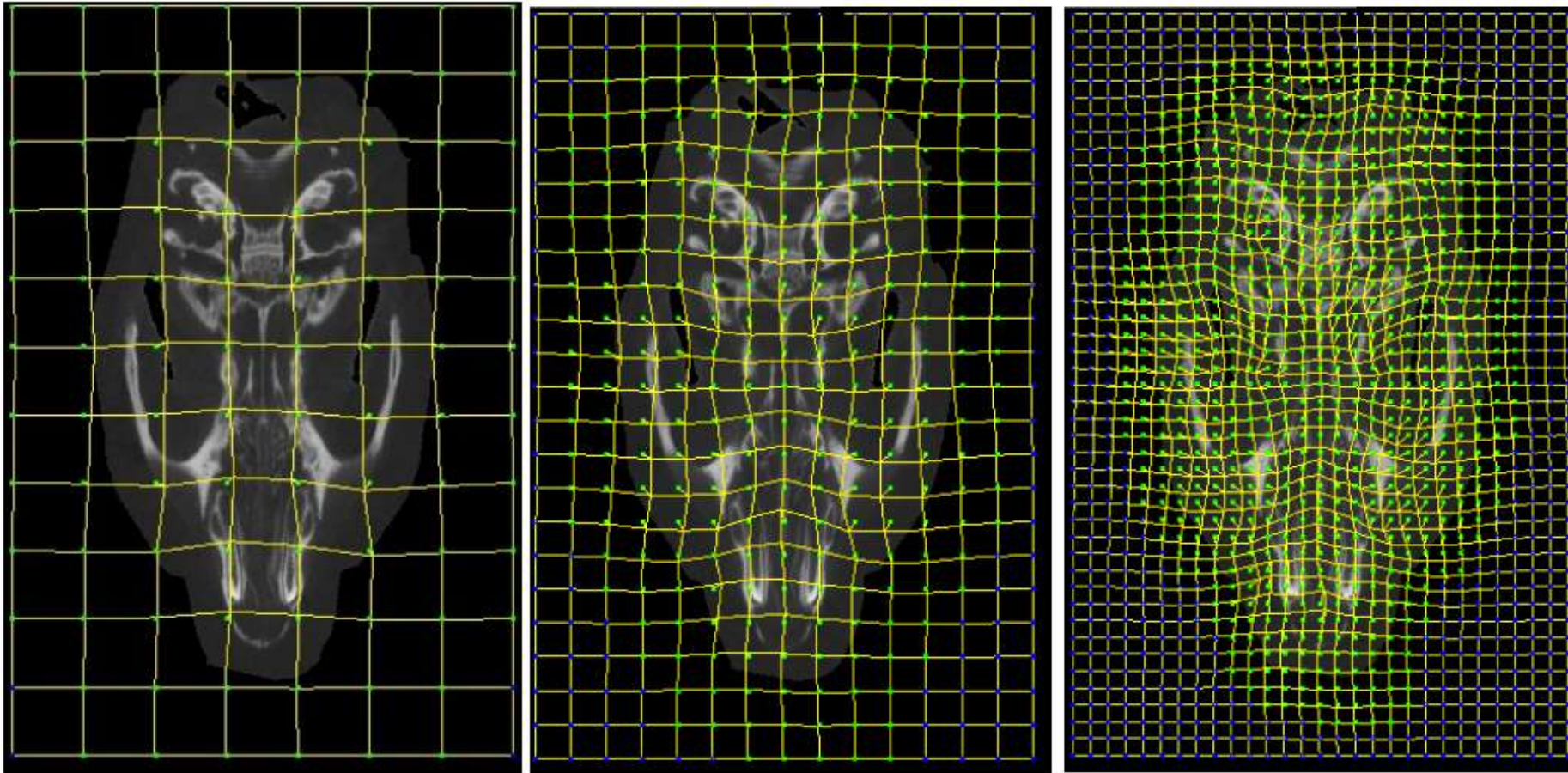


The Need for Smooth Warp Fields

- **Observation:** Permitting one vector per voxel would provide too much flexibility
 - **Problem 1:** The degrees of freedom in the transformation exceed the available image information
 - **Problem 2:** The following scheme achieves a good match in terms of intensities, but not a meaningful registration:
 - Specify displacement vectors that map the i^{th} brightest voxel in the floating image to the i^{th} brightest voxel in the reference
- **Popular solution:** Create a smooth deformation field from displacement vectors on a coarser grid

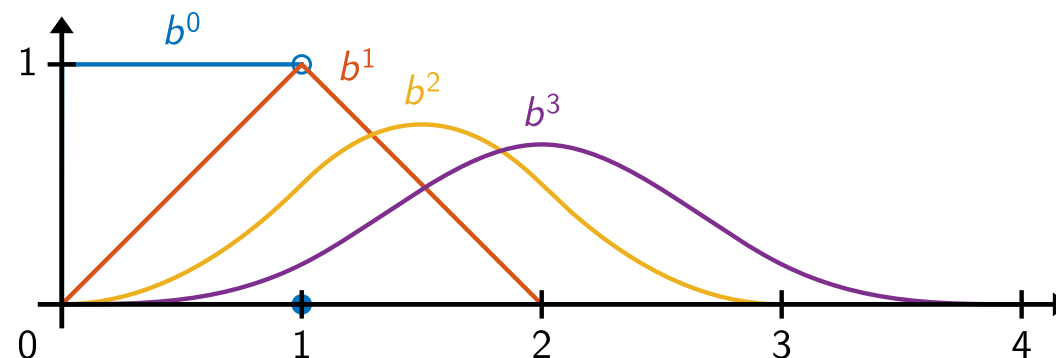
Deformation Fields: Smooth Parametrizations

- **Illustration:** Grid resolution determines the degrees of freedom (smoothness / flexibility) in nonlinear registration



Definition: Cardinal B-spline

- **Recursive definition** of a cardinal B-spline b^p of degree $p \in N_0$:
 - **Base case:** A B-spline b^0 of degree zero is the indicator function on the half-open unit interval $[0,1)$
 - **Definition:** b^p results from the convolution $b^p = b^0 * b^{p-1}$



- **Properties of b^p :**

1. Piecewise polynomials, degree p
2. Smoothness: $b^p \in C^{p-1}$
3. Compact support: $b^p = 0$ outside $[0, p + 1)$

Cubic B-splines in registration:
Efficient evaluation and differentiation
 C^2 smoothness for optimization
Sparse J_h with sum-of-squares cost

Uniform and Tensor Product B-Splines

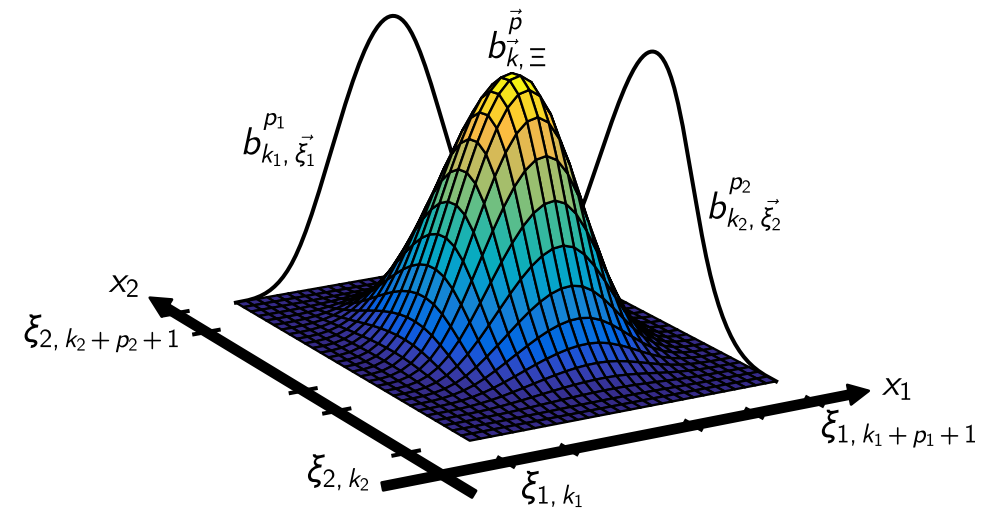
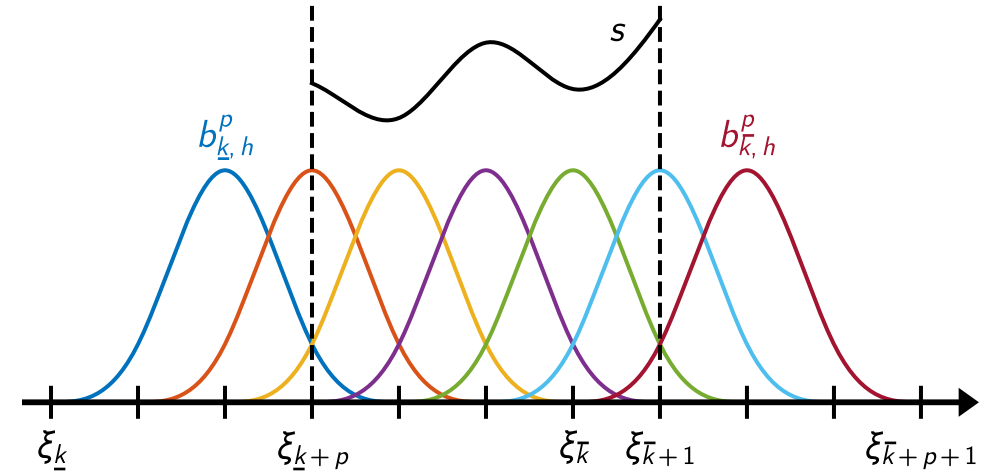
- **Uniform B-splines** arise from cardinal B-splines via shifting and scaling:

$$b_{k,h}^p(x) = b^p\left(\frac{x}{h} - k\right)$$

- In our context, represent the influence of vector at position k with grid spacing h
- Duplicating values is a simple way to deal with the boundary

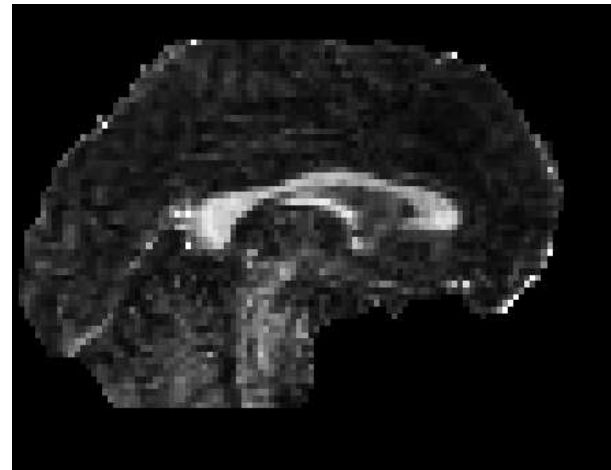
- **Multi-dimensional B-Splines** arise from the point-wise product (“tensor product”)

$$b_{\mathbf{k}}^{\mathbf{p}}(\mathbf{x}) = \prod_{t=1}^d b_{k_t}^{p_t}(x_t)$$

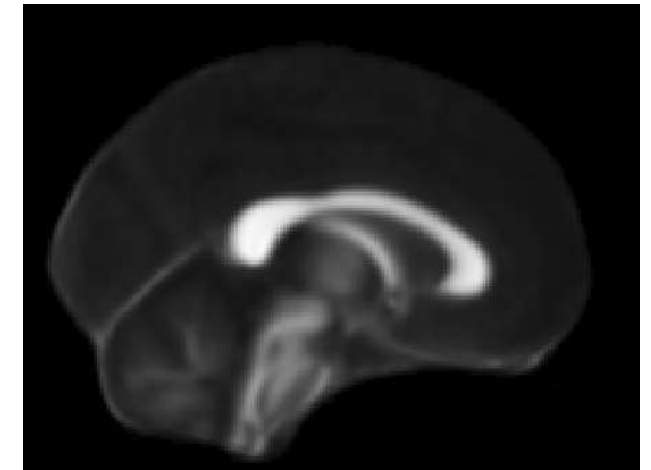


Reminder: Overall Normalization Pipeline

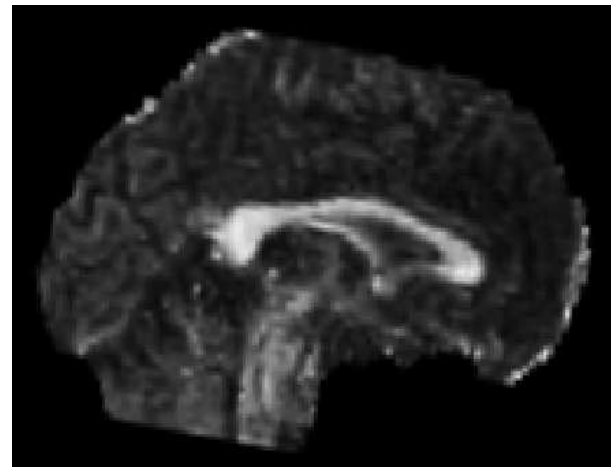
- **Affine („linear“) registration first**
 - Mostly achieves correct position, orientation, and scale
- **Nonlinear registration second**
 - required to account for local deformations



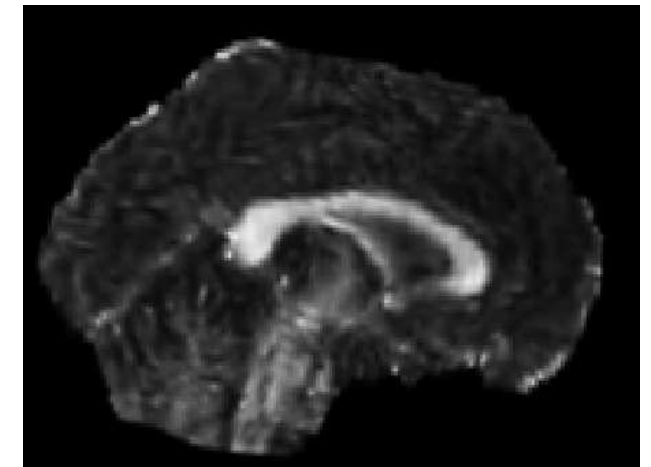
Single Subject Scan



Template



Affine Transformation



Nonlinear Warp

Nonlinear Registration: Assumptions

- A common assumption in nonlinear registration is that **affine transformations** have already been accounted for
 - It is safe to neglect issues with partial overlaps
 - Displacement vectors at boundary may be constrained to be tangential to boundary (or zero)
- Nonlinear registration is most commonly performed **intra-modally**, e.g., with sum of squares cost function
 - To normalize different modalities:
 - perform linear inter-modal registration within-subject
 - compose with nonlinear intra-modal registration
- **Challenge:** Optimization space too large for derivative-free methods
 - **But:** Feasible to compute derivatives of cost under the assumptions above

Gradient of Sum-of-Squares Cost

- The **sum of squares** cost function can be written as $f(\mathbf{w}) = \mathbf{h}(\mathbf{w})^T \mathbf{h}(\mathbf{w})$ with $\mathbf{h}(\mathbf{w}) = \mathbf{y} - \mathbf{T}(\mathbf{x}|\mathbf{w})$
 - $\mathbf{y}$ =vectorized fixed image; $\mathbf{T}(\mathbf{x}|\mathbf{w})$ transforms moving image $\mathbf{x}$ with parameters $\mathbf{w}$

- The j th coefficient of ∇f is

$$\frac{\partial f}{\partial w_j} = -2 \sum_{p=1}^N ([\mathbf{y}]_p - [\mathbf{T}(\mathbf{x}|\mathbf{w})]_p) \frac{\partial [\mathbf{T}(\mathbf{x}|\mathbf{w})]_p}{\partial w_j}$$

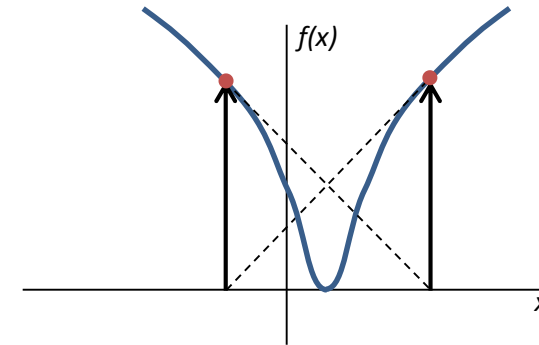
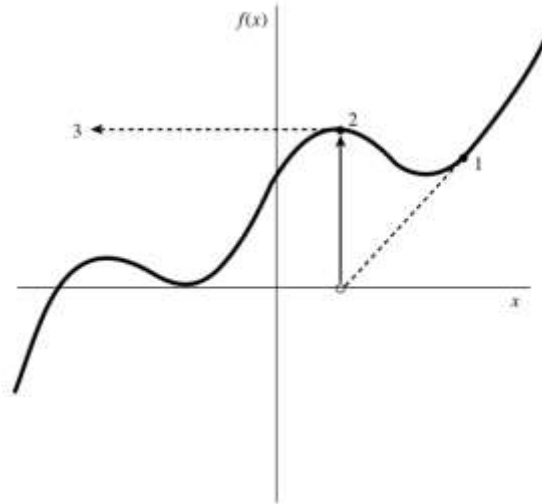
- Gradient of f in terms of Jacobian $\mathbf{J}_h$:

$$\nabla f = 2\mathbf{J}_h^T \mathbf{h}(\mathbf{w})$$

Gradient Descent

- **Idea:** Use $-\nabla f(\mathbf{w})$ as a search direction
- *Simple:* Multiply gradient with a fixed stepsize $1/\lambda$ and subtract it from the current position: $\mathbf{w}_{k+1} = \mathbf{w}_k - \frac{1}{\lambda} \nabla f(\mathbf{w}_k)$

– Failure cases:



- *Safe:* Perform 1D optimization (“line search”) along the direction of $-\nabla f(\mathbf{w})$

Newton's Method

- **Newton's method** attempts to “jump” to the optimum of $f(\mathbf{w})$ based on a second-order Taylor approximation around the current guess $\mathbf{w}_k$

$$f(\mathbf{w}) \approx f(\mathbf{w}_k) + \nabla f(\mathbf{w}_k)(\mathbf{w} - \mathbf{w}_k) + \frac{1}{2}(\mathbf{w} - \mathbf{w}_k)^T \mathbf{H}(\mathbf{w}_k)(\mathbf{w} - \mathbf{w}_k)$$

- This results in the approximation

$$\nabla f(\mathbf{w}) \approx \nabla f(\mathbf{w}_k) + \mathbf{H}(\mathbf{w}_k)(\mathbf{w} - \mathbf{w}_k)$$

- Setting it to zero results in the update equation

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \mathbf{H}^{-1}(\mathbf{w}_k) \nabla f(\mathbf{w}_k)$$

Hessian of Sum-of-Squares Cost

- **Reminder:** The **sum of squares** cost function can be written as $f(\mathbf{w}) = \mathbf{h}(\mathbf{w})^T \mathbf{h}(\mathbf{w})$ with $\mathbf{h}(\mathbf{w}) = \mathbf{y} - \mathbf{T}(\mathbf{x}|\mathbf{w})$
 - $\mathbf{y}$ =vectorized fixed image; $\mathbf{T}(\mathbf{x}|\mathbf{w})$ transforms moving image $\mathbf{x}$ with parameters $\mathbf{w}$
- **Reminder:** The j th coefficient of ∇f is $\nabla f = 2\mathbf{J}_h^T \mathbf{h}(\mathbf{w})$

$$\frac{\partial f}{\partial w_j} = -2 \sum_{p=1}^N ([\mathbf{y}]_p - [\mathbf{T}(\mathbf{x}|\mathbf{w})]_p) \frac{\partial [\mathbf{T}(\mathbf{x}|\mathbf{w})]_p}{\partial w_j}$$

- The (i,j) th coefficient of its Hessian is

$$\frac{\partial^2 f}{\partial w_i \partial w_j} = 2 \sum_{p=1}^N \frac{\partial [T(\mathbf{x}|\mathbf{w})]_p}{\partial w_i} \frac{\partial [T(\mathbf{x}|\mathbf{w})]_p}{\partial w_j} - \cancel{([\mathbf{y}]_p - [T(\mathbf{x}|\mathbf{w})]_p) \frac{\partial^2 [T(\mathbf{x}|\mathbf{w})]_p}{\partial w_i \partial w_j}}$$

$$\mathbf{H} \approx 2\mathbf{J}_h^T \mathbf{J}_h$$

will be neglected

Gauss-Newton Algorithm

- Substitution into the standard Newton iteration

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \mathbf{H}^{-1}(\mathbf{w}_k) \nabla f(\mathbf{w}_k)$$

yields the **Gauss-Newton iteration**

$$\mathbf{w}_{k+1} = \mathbf{w}_k - (\mathbf{J}_h^T \mathbf{J}_h)^{-1} \mathbf{J}_h^T \mathbf{h}(\mathbf{w}_k)$$

- Benefits of this approximation:
 - Unlike $\mathbf{H}$, $2\mathbf{J}_h^T \mathbf{J}_h$ is positive (semi-)definite by construction
 - increases stability
 - Saves cost of computing second derivatives
 - $\mathbf{J}_h$ is high-dimensional, but sparse
 - derivative of individual pixel depends on few deformation parameters
- *But:* Might not converge if $\mathbf{w}_0$ is too far from optimum

Levenberg's Method

- **Levenberg's method** interpolates between
 - **Gradient descent** away from optimum
 - Guarantees progress with sufficiently small stepsize
 - **Gauss-Newton** close to optimum (faster convergence)
- Achieved via the **regularized / damped update**

$$\mathbf{w}_{k+1} = \mathbf{w}_k - (\mathbf{J}_h^T \mathbf{J}_h + \lambda \mathbf{I})^{-1} \mathbf{J}_h^T \mathbf{h}(\mathbf{w}_k)$$

- Approximates gradient descent for large λ
- Approximates Gauss-Newton for $\lambda \approx 0$
- Different heuristics for choice of λ . Example:
 - Start with a moderate value, e.g., $\lambda = 0.001$
 - Accept steps that improve $f(\mathbf{w})$, decrease λ by factor $\rho \in (0,1)$
 - If a step fails to improve $f(\mathbf{w})$, increase λ by $1/\rho$ and try again

Levenberg-Marquardt

- Gradient descent might make **unequal progress** along different dimensions if individual parameters w_j are scaled differently (e.g., have different physical units)

– *Idea:* Stepsize $\left[\lambda \frac{\partial^2 f}{\partial w_j \partial w_j} \right]^{-1}$ for gradient component $\frac{\partial f}{\partial w_j}$ makes progress scale invariant (changes in scale of w_j cancel out)

- Results in **Levenberg-Marquardt update:**

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \left(\mathbf{J}_h^T \mathbf{J}_h + \lambda \text{diag}(\mathbf{J}_h^T \mathbf{J}_h) \right)^{-1} \mathbf{J}_h^T \mathbf{h}(\mathbf{w}_k)$$

– $\text{diag}(\mathbf{J}_h^T \mathbf{J}_h)$ preserves diagonal, sets off-diagonal elements to zero

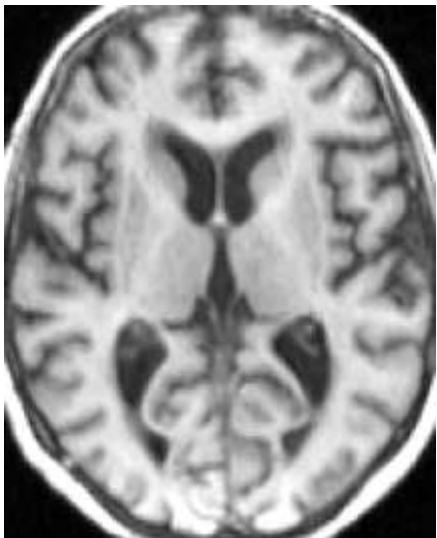
Summary: Nonlinear Registration

- **Nonlinear registration** involves a huge number of parameters. Their optimization requires guidance from derivatives
 - Smooth B-Spline-based deformation fields permit differentiation
 - Common to rely on sum-of-squares, ignore partial overlaps
 - Assumes that affine registration has already been achieved
- **Gradient descent** with sufficiently small steps always makes progress, can be made scale invariant using Hessian diagonal
- **Gauss-Newton** provides faster convergence close to the optimum
 - Exploits sum-of-squares structure to avoid complications from second derivatives in Newton's method
- **Levenberg-Marquardt** adaptively interpolates between gradient descent and Gauss-Newton

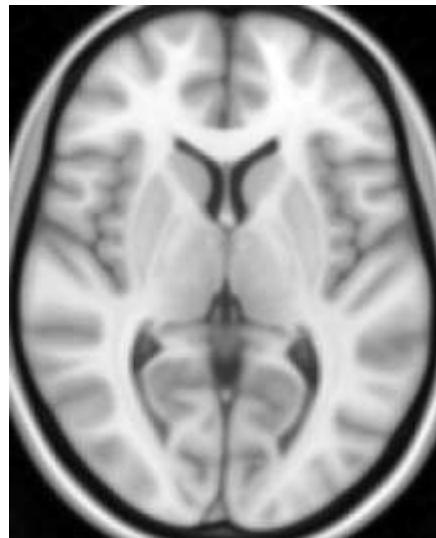
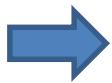
4.7 Regularizing Nonlinear Registration

Motivation

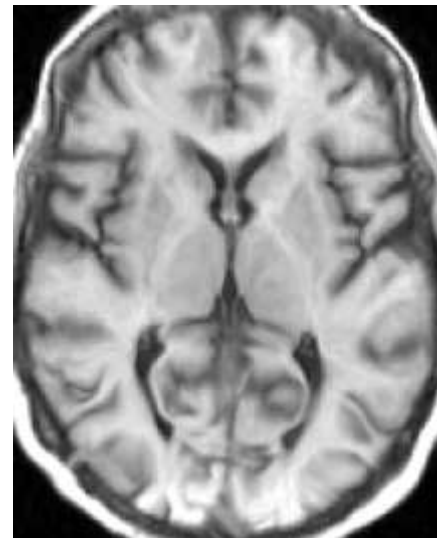
- **Problem:** B-spline interpolation ensures smooth deformation vector fields, but still permits drastic and unrealistic deformations
 - Including “collapse” of a local neighborhood into a single point or even a local orientation reversal (e.g., up-down or left-right)



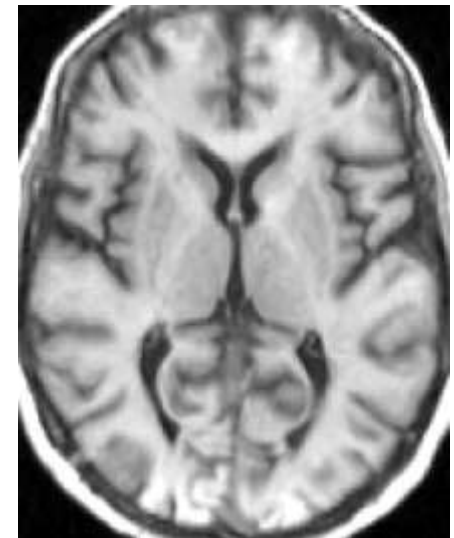
Linear Registration



Template



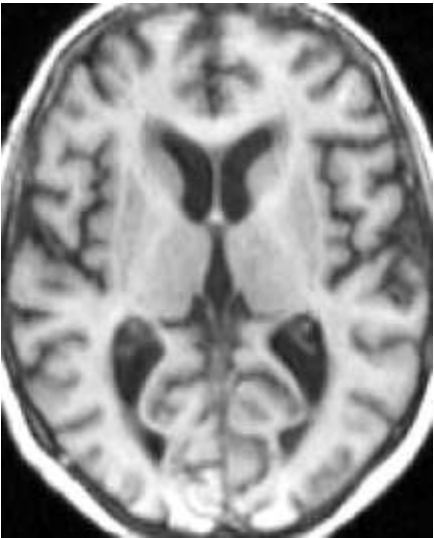
Low Regularity



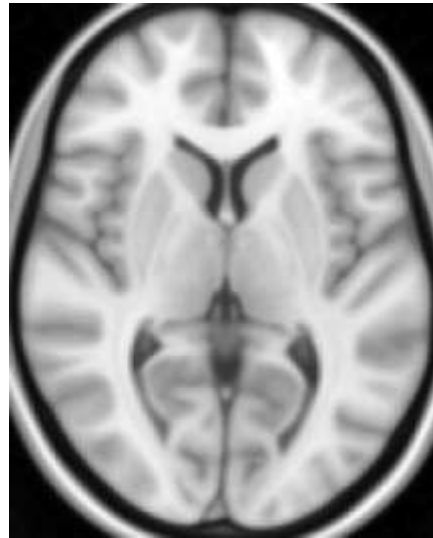
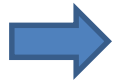
Medium Regularity

Motivation

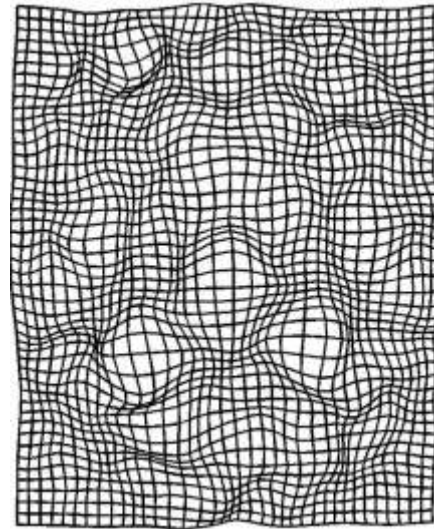
- **Problem:** B-spline interpolation ensures smooth deformation vector fields, but still permits drastic and unrealistic deformations
 - Including “collapse” of a local neighborhood into a single point or even a local orientation reversal (e.g., up-down or left-right)



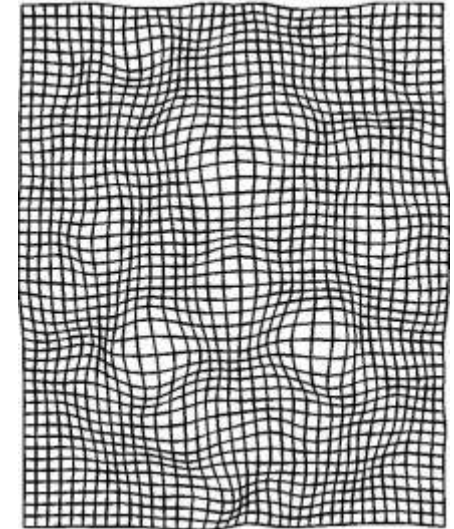
Linear Registration



Template



Low Regularity



Medium Regularity

Vector Field Jacobian

- *Definition:* The **Jacobian matrix $\mathbf{J}$** of an N -D vector field is the $N \times N$ matrix of all first-order partial derivatives

$$\mathbf{v}(x, y) = \begin{pmatrix} u(x, y) \\ v(x, y) \end{pmatrix}$$

2D vector field

$$\mathbf{J}(x, y) = \begin{pmatrix} u_x & u_y \\ v_x & v_y \end{pmatrix}$$

Jacobian of a 2D vector field

- *Definition:* The **Jacobian determinant $|\mathbf{J}|$** is the determinant of the Jacobian matrix. It tells us what happens to local volume elements
 - $|\mathbf{J}| > 1$: local expansion
 - $|\mathbf{J}| < 1$: local compression
 - $|\mathbf{J}| = 0$: non-invertible
 - $|\mathbf{J}| < 0$: orientation-reversal

Elastic Registration

- **Elastic registration** seeks a tradeoff between image similarity and regularity of the transformation
 - Regularization based on the Jacobian matrix
- *Example: PASHA algorithm (Cachier 2003)*
 - “Pair-And-Smooth, Hybrid energy based Algorithm”
 - Given fixed image F and moving M , it minimizes the following energy:

$$E(\varphi, \psi) = \underbrace{\|F(x) - M(\psi(x))\|_2^2}_{\text{L2 Similarity}} + \underbrace{\sigma \|\varphi(x) - \psi(x)\|_2^2}_{\text{Coupling}} + \underbrace{\sigma \lambda \|J_\varphi(x)\|_F^2}_{\text{Regularity}}$$

transformation
of M

“correspondences”,
hidden variables that
simplify optimization

Optimizing PASHA

- Optimization in PASHA iterates two steps:
 1. Minimize $\|F(x) - M(\psi(x))\|_2^2 + \sigma\|\varphi(x) - \psi(x)\|_2^2$ with respect to ψ by performing a line search along its negative gradient
$$2[F(x) - M(\psi(x))] \nabla M(\psi(x)) + 2\sigma[\varphi(x) - \psi(x)]$$
 2. Minimize $\|\varphi(x) - \psi(x)\|_2^2 + \lambda\|J_\varphi(x)\|_F^2$ with respect to φ . This can be done in closed form by a convolution: $\varphi = K_\lambda * \psi$

Note: Exact kernel K_λ and its derivation are not presented here.

Related Problem: Optical Flow

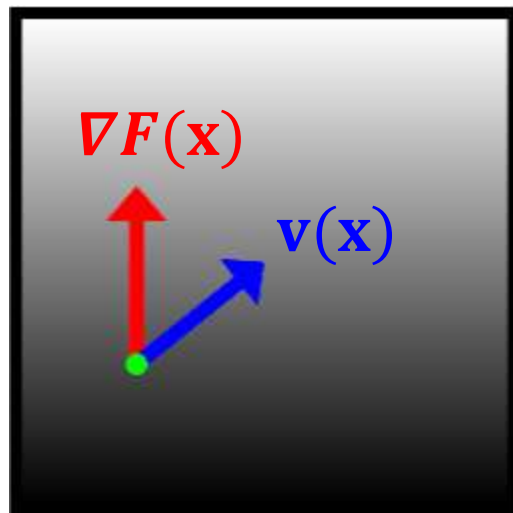


Images from Blender Foundation / bigbuckbunny.org

Optical Flow

- **Goal:** Given two frames of a movie, find the displacement $\mathbf{v}(\mathbf{x})$ at each pixel position $\mathbf{x}$ from one to the next
- **Assumption:** Pixels move, but do not change their intensity
- First-order approximation:

$$M(\mathbf{x}) = F(\mathbf{x}) - \langle \mathbf{v}(\mathbf{x}), \nabla F(\mathbf{x}) \rangle$$



Note: We only observe motion along the gradient. This is known as the *aperture problem*.

Solving the Aperture Problem

- To solve the aperture problem, we pick the displacement vector $\mathbf{v}(\mathbf{x})$ with smallest possible norm: $\mathbf{v}(\mathbf{x}) = \lambda \nabla F(\mathbf{x})$
- It then follows from $M(\mathbf{x}) = F(\mathbf{x}) - \langle \mathbf{v}(\mathbf{x}), \nabla F(\mathbf{x}) \rangle$ that

$$M(\mathbf{x}) = F(\mathbf{x}) - \lambda \|\nabla F(\mathbf{x})\|_2^2$$

$$\lambda = \frac{F(\mathbf{x}) - M(\mathbf{x})}{\|\nabla F(\mathbf{x})\|_2^2}$$

$$\mathbf{v}(\mathbf{x}) = \frac{F(\mathbf{x}) - M(\mathbf{x})}{\|\nabla F(\mathbf{x})\|_2^2} \nabla F(\mathbf{x})$$

Demons Regularization

- **Demons Regularization** avoids numerical problems in

$$\mathbf{v}(\mathbf{x}) = \frac{F(\mathbf{x}) - M(\mathbf{x})}{\|\nabla F(\mathbf{x})\|_2^2} \nabla F(\mathbf{x})$$

that arise when $\|\nabla F(\mathbf{x})\|^2 \approx 0$:

$$\mathbf{v}_D(\mathbf{x}) = \frac{F(\mathbf{x}) - M(\mathbf{x})}{\|\nabla F(\mathbf{x})\|_2^2 + (F(\mathbf{x}) - M(\mathbf{x}))^2} \nabla F(\mathbf{x})$$

- $\mathbf{v}_D(\mathbf{x}) \rightarrow 0$ if either $\|\nabla F(\mathbf{x})\| \rightarrow 0$ or $(F(\mathbf{x}) - M(\mathbf{x})) \rightarrow 0$
- If $\|\nabla F(\mathbf{x})\|_2^2 + (F(\mathbf{x}) - M(\mathbf{x}))^2 < \epsilon : \mathbf{v}_D(\mathbf{x}) := \mathbf{0}$

Effect of Demons Regularization

- **Observation:** Demons regularization keeps correspondences in a local neighborhood by bounding $\|\mathbf{v}_D\|$:

$$\begin{aligned}\|\mathbf{v}_D(\mathbf{x})\| &= \frac{|F(\mathbf{x}) - M(\mathbf{x})| \|\nabla F(\mathbf{x})\|}{\|\nabla F(\mathbf{x})\|_2^2 + (F(\mathbf{x}) - M(\mathbf{x}))^2} \\ &= \frac{GM^2(|F(\mathbf{x}) - M(\mathbf{x})|, \|\nabla F(\mathbf{x})\|)}{2 QM^2(|F(\mathbf{x}) - M(\mathbf{x})|, \|\nabla F(\mathbf{x})\|)} \leq 0.5\end{aligned}$$

- where GM=geometric mean, QM=quadratic mean
- Makes use of HM-GM-AM-QM inequalities

Demons Algorithm for Registration

- Elastic registration with the Demons algorithm (Thirion 1998) iterates these steps:
 1. Compute regularized optical flow $\bar{\psi}(x)$ between fixed image $F(x)$ and moving image $M(\psi(x))$.
 2. Add flow vectors to current displacement field to obtain $\hat{\psi}(x) = \psi(x) + \bar{\psi}(x)$
 3. Perform Gaussian smoothing to obtain $\varphi(x) = G_s * \hat{\psi}(x)$.
The result will serve as the new $\psi(x)$ in the next iteration

Justification: “Demons” are sitting on the pixels and locally move the image based on intensity differences.
Comparison to PASHA permits a better understanding.

PASHA vs. Demons

Similarities between the Demons algorithm and PASHA:

- Both search correspondences based on the image gradient, and regularize them to remain in the current neighborhood.
 - negative gradient in PASHA
$$2[F(x) - M(\psi(x))] \nabla M(\psi(x)) + 2\sigma[\varphi(x) - \psi(x)]$$
 - Demons' optical flow vector
$$\frac{F(x) - M(\psi(x))}{\|\nabla F(x)\|_2^2 + (F(x) - M(\psi(x)))^2} \nabla F(x)$$
- Both apply a convolution to regularize these correspondences.

Limitations of Elastic Registration (Cars)

- The Jacobian penalty was added to make the transformation “reasonable”, but can prevent (desired) large deformations



Fixed



Moving



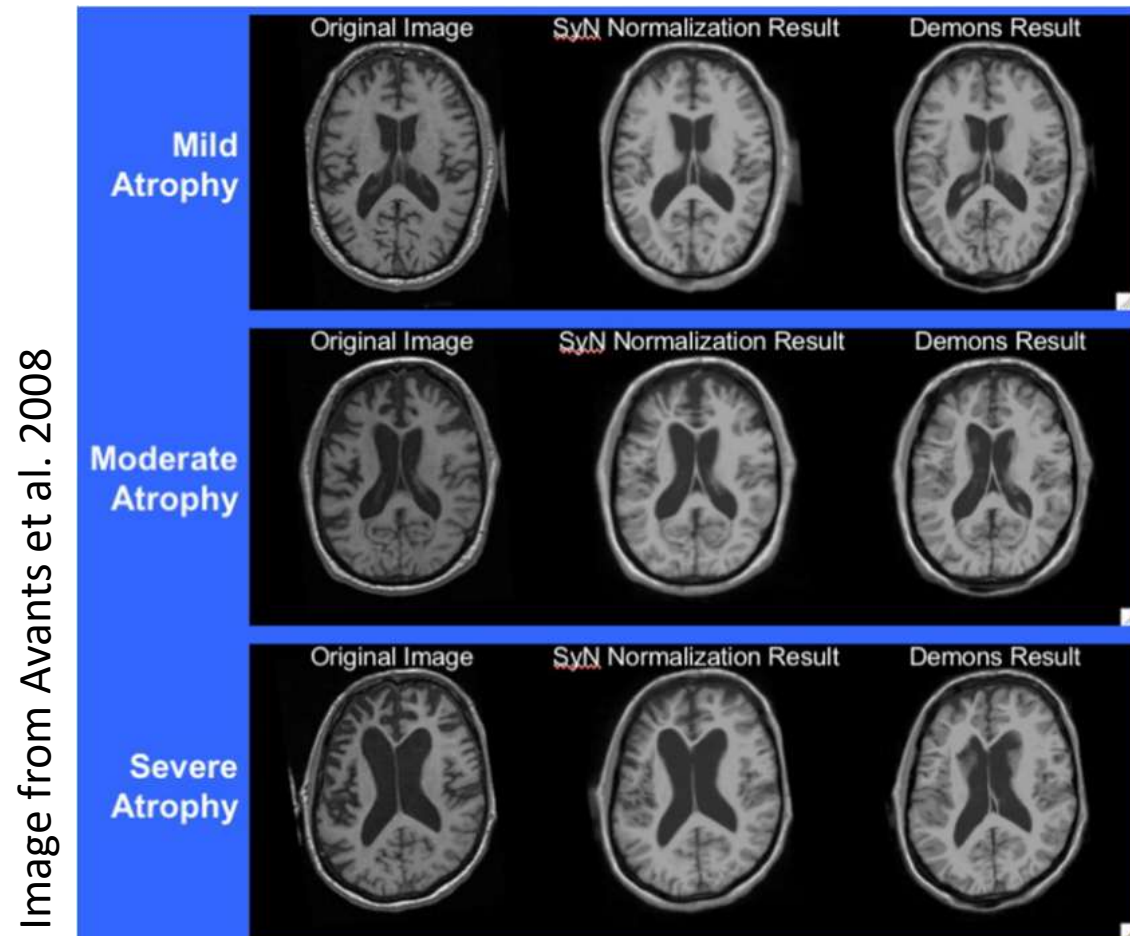
Affine

Diffeomorphisms

- Criteria for a transformation to be “reasonable” may include:
 1. Differentiability of the vector field
 2. Invertibility of the transformation
 3. Differentiability of the inverse
- *Definition:* A **diffeomorphism** is a transformation that fulfills all of the above
- Basis of **LDDMM** = Large Deformation Diffeomorphic Metric Mapping

Limitations of Elastic Registration (Brains)

- Severe atrophy is handled better with diffeomorphic than with elastic registration:



Intuition: Time-Varying Velocity Fields

- You can think of the transformation in the LDDMM framework as happening continuously over time t



$t=0$

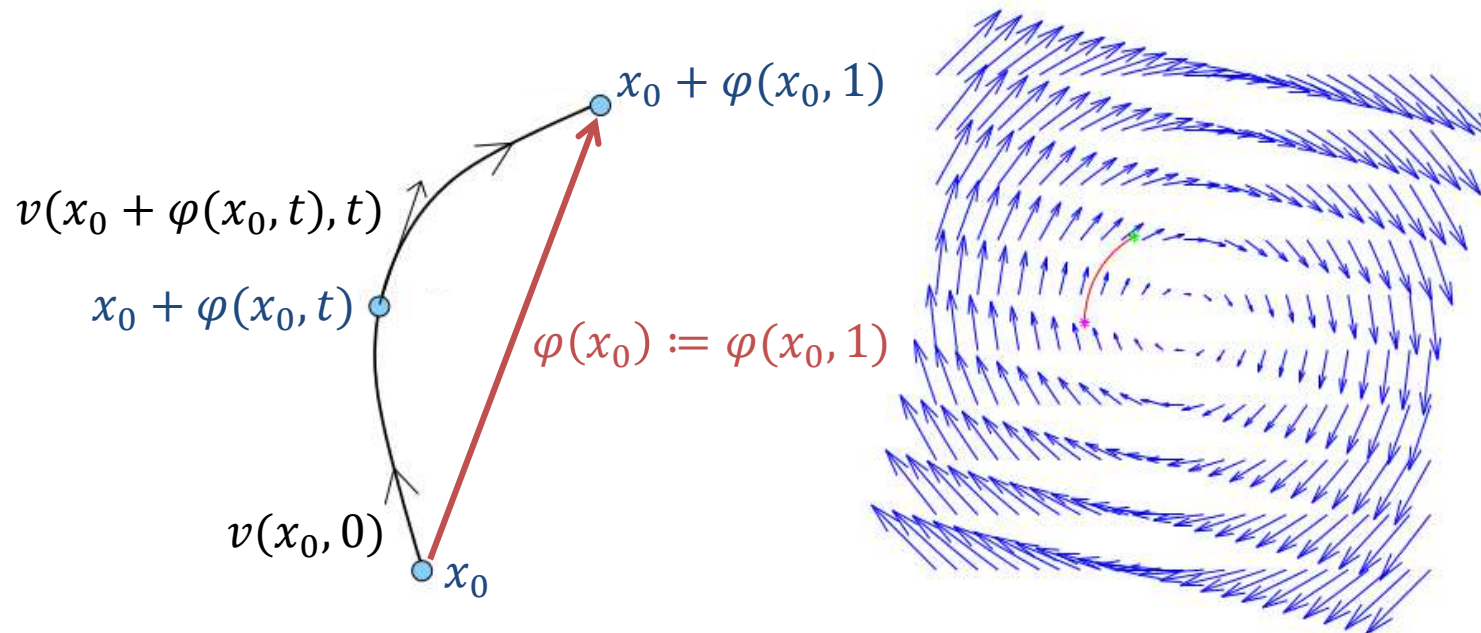
$t=0.5$

$t=1$

From Time-Varying Velocity Fields to Deformations

- A diffeomorphic **displacement vector field** $\varphi(x)$ is obtained by integrating a smooth time-varying **velocity vector field** $v(x,t)$:

$$\frac{\partial \varphi(x, t)}{\partial t} = v(x + \varphi(x, t), t)$$

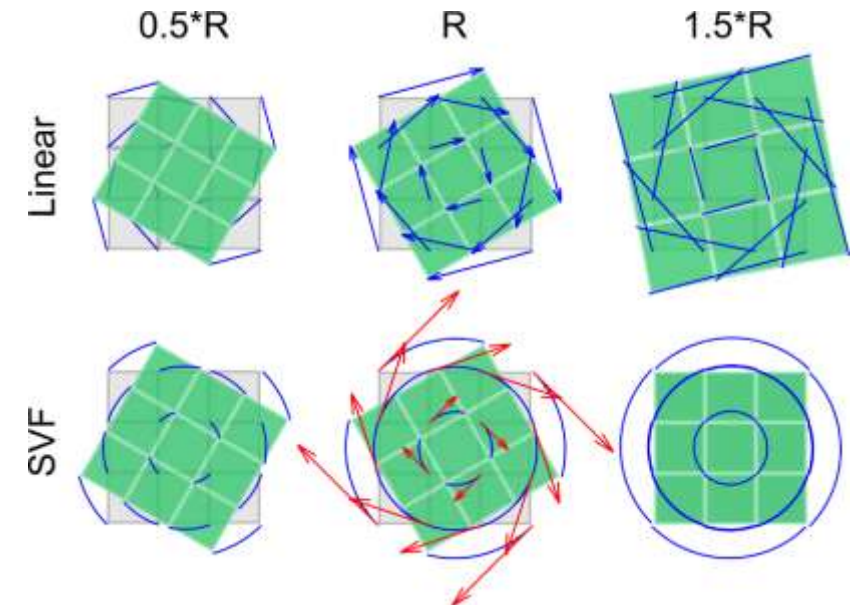


Properties of Velocity Field Representation

1. **Differentiability** of φ follows from smoothness of v

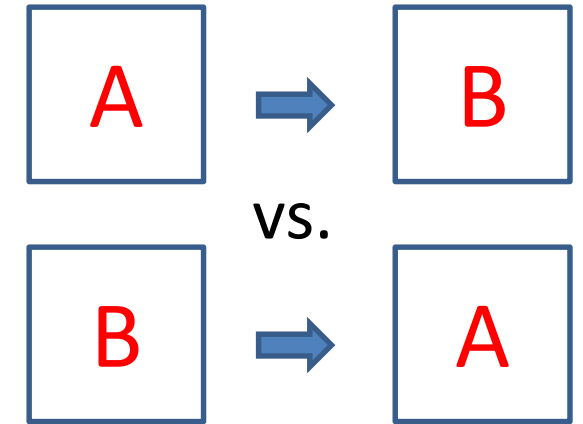
2. **Differentiable inverse** of φ given by integrating $-v$

- By definition: $\varphi(x, 0) = \text{Id}$
- By convention: $\varphi(x) := \varphi(x, 1)$
- By construction: $\varphi^{-1}(\varphi) = \text{Id}$
- Interpolation / extrapolation results more useful if we work with velocity vector fields than with displacements



Symmetric Diffeomorphic Registration

- Formulating registration symmetrically with respect to both images enforces consistency
 - No assignment of fixed or moving image
 - Example:* SyN by Avants et al. 2008:



$$\inf_{\varphi_1} \inf_{\varphi_2} \int_{t=0}^{0.5} \{ \|v_1(x, t)\|_L^2 + \|v_2(x, t)\|_L^2 \} dt \\ + \int_{\Omega} |I_1(\varphi_1(0.5)) - I_2(\varphi_2(0.5))|^2 d\Omega$$

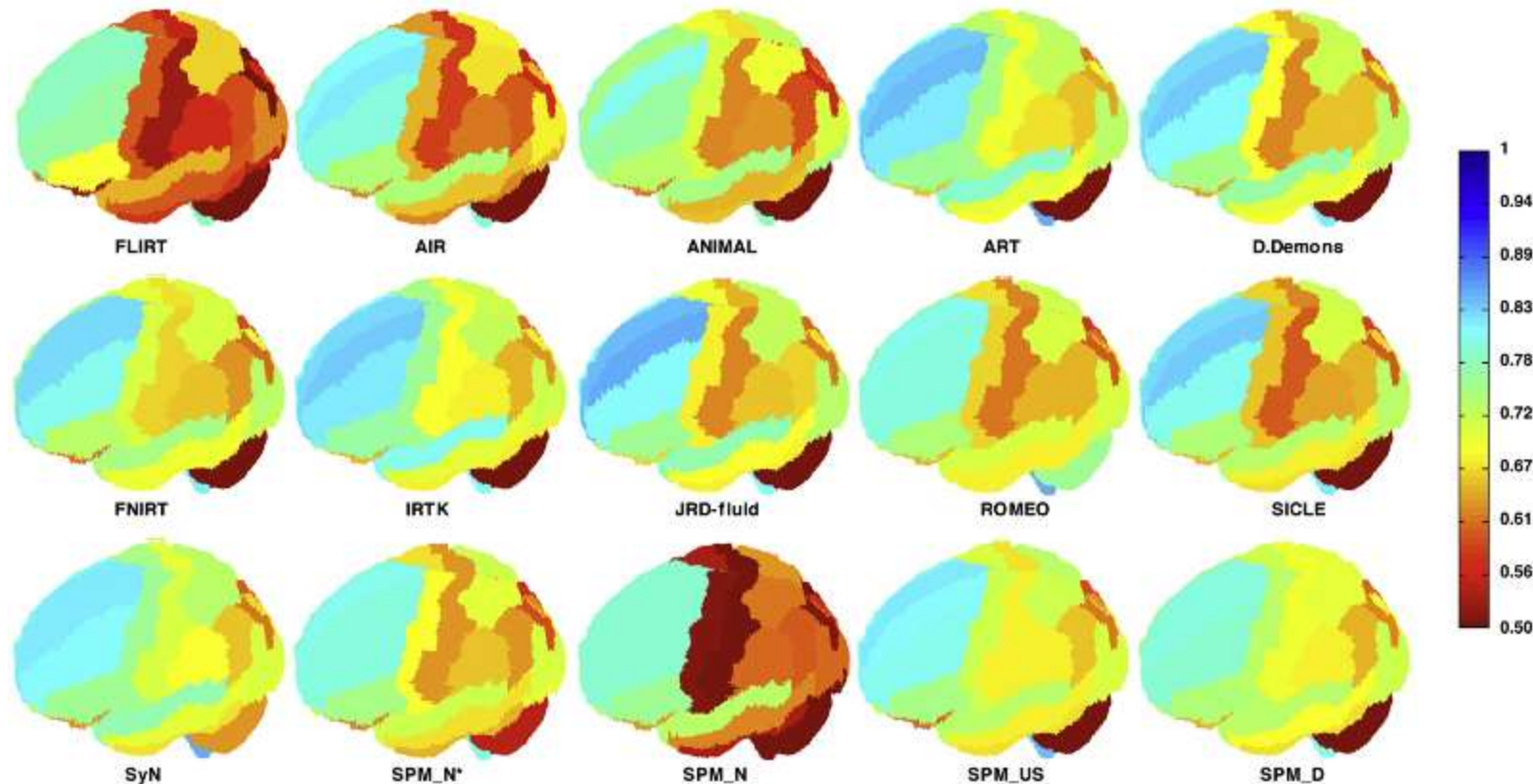
- Where $\|v(x)\|_L^2 := \int_{\Omega} \|Lv(x)\|^2 d\Omega$ with $L = a\nabla^2 + b\text{Id}$ penalizes large or irregular velocity vector fields
- If desired, we can still map I_1 to I_2 using $\varphi_2^{-1} \circ \varphi_1$

Atlas-based Labeling

- **Labeling:** Important application of registration
- **Problem:** Manual labeling is expensive
 - It takes at least one month of training to learn how to label with acceptable reproducibility
 - When done carefully, labeling a single MRI brain scan can take an expert between one and two weeks
- **Templates** can be used to transfer expert labels to new data:
 1. Normalize images, apply same transformation to labels
 2. In each template voxel, estimate label probabilities
 3. Register new brain to template, look up labels

Labeling to Evaluate Normalization

- In 2009, a big study by Klein et al. compared 14 nonlinear registration methods based on their success in transferring manual labels
 - Target overlap: What fraction of each region has been labeled correctly?



Limitations of Diffeomorphic Registration

Some caveats from Avants et al. 2008:

- “It is not yet known the extent to which brains of different individuals, when represented as magnetic resonance images, **are diffeomorphic to each other**. This problem is even less well understood with elderly and patient brains.”
- “Although the quality of our results are reasonable, by some standards, these algorithms, as applied here, **cannot claim to accurately reproduce manual labelings**. The reasons for this are well-known. The primary reason is that expert knowledge is not directly encoded in these methods.”

Summary: Regularizing Nonlinear Registration

- Nonlinear registration is an ill-posed problem that requires **regularization**.
 - The type and strength of regularization can significantly affect the findings in group studies (this will come up again in Chapter 6).
- **Elastic registration**
 - Penalizes strong deformations via Jacobian
 - *Examples:* PASHA and the Demons algorithm
- **Diffeomorphic registration (LDDMM)**
 - Diffeomorphisms parameterized using velocity vector fields
 - SyN algorithm: Symmetric, no “fixed” and “moving” images

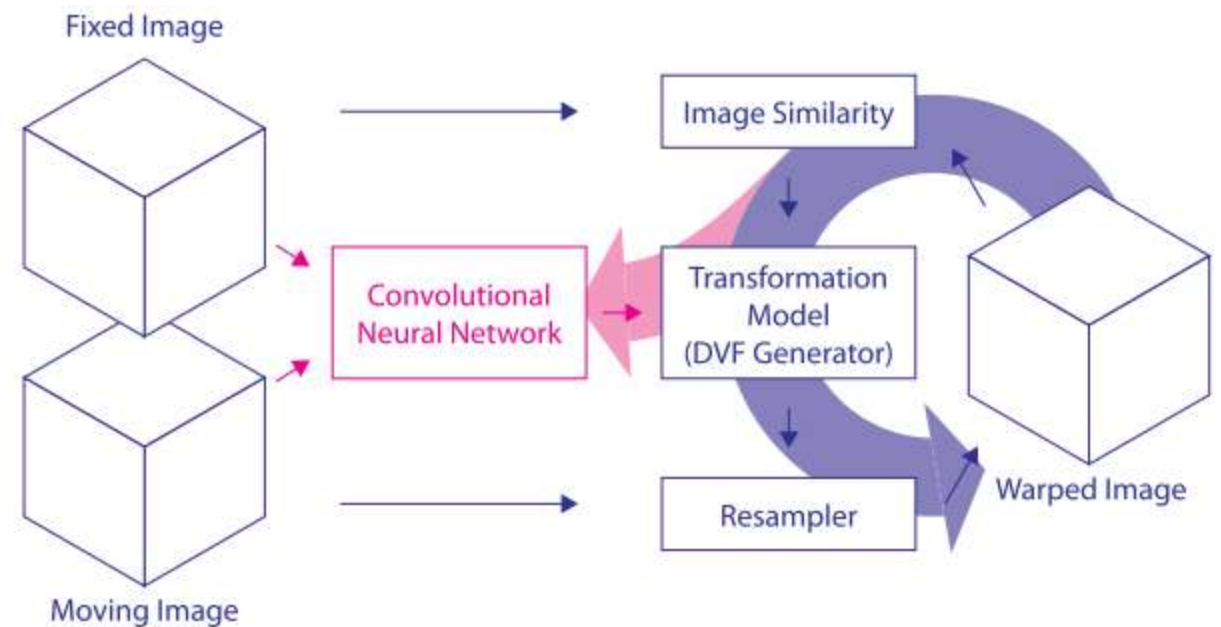
4.8 Neural Network Based Image Registration

Motivation

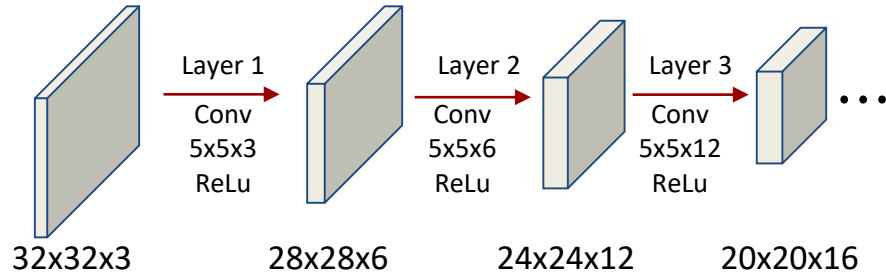
- **Neural networks** can be trained to map a given image pair to
 - Parameters for an affine registration
 - B-spline control points for a displacement vector field
 - Velocity vectors for diffeomorphic image registration
- We will consider **CNNs** (known from Chapter 3.7)
- **Benefits** of learning-based approach:
 - Much faster, avoids costly optimization for each new image pair
 - Local minima “smooth out” when training over a large set of images
 - Can be taught to handle strong initial misalignments

Unsupervised Affine and Deformable Image Registration

- Approach by [de Vos et al. 2019] remains close to traditional intensity-based registration
 - Rather than optimize transformation directly, optimize **weights of a CNN** that maps image pairs to required transformation
 - **Cost function** remains similar
 - Normalized cross-correlation
 - Regularized via bending energy, keeps displacement vector fields smooth
 - Registration requires a large **receptive field**



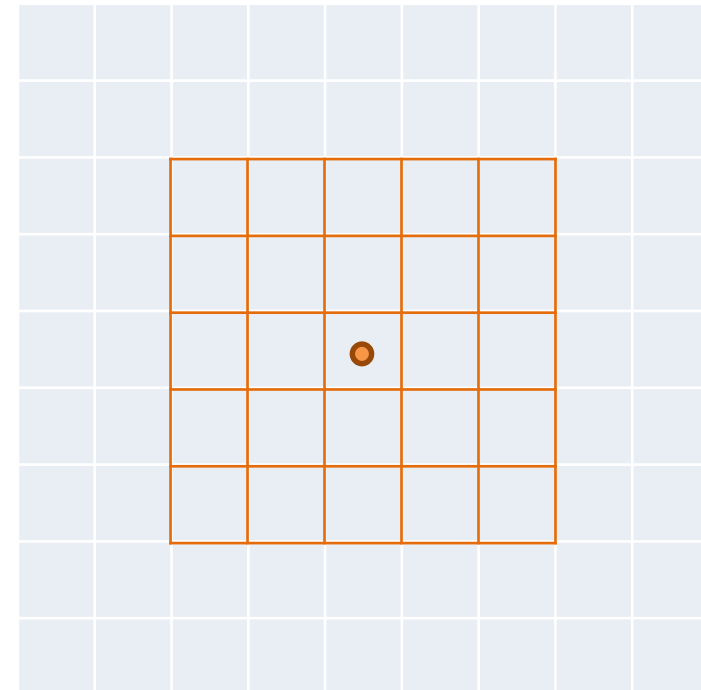
Receptive Field



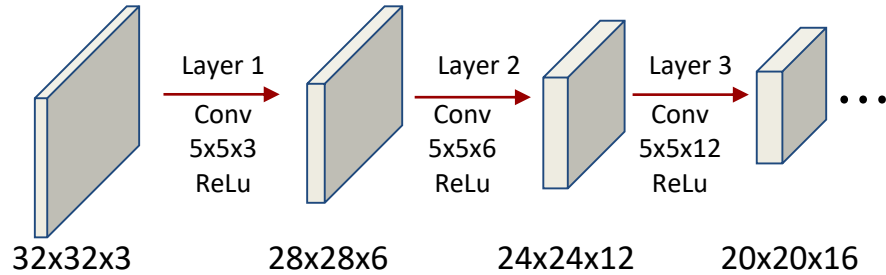
The receptive field of a neuron is defined as the region in the input volume that the neuron is looking at (i.e. is affected by).

The receptive field in Layer 1 is 5x5.
How about in Layer 2 and Layer 3?

1st convolution layer
2nd convolution layer



Receptive Field

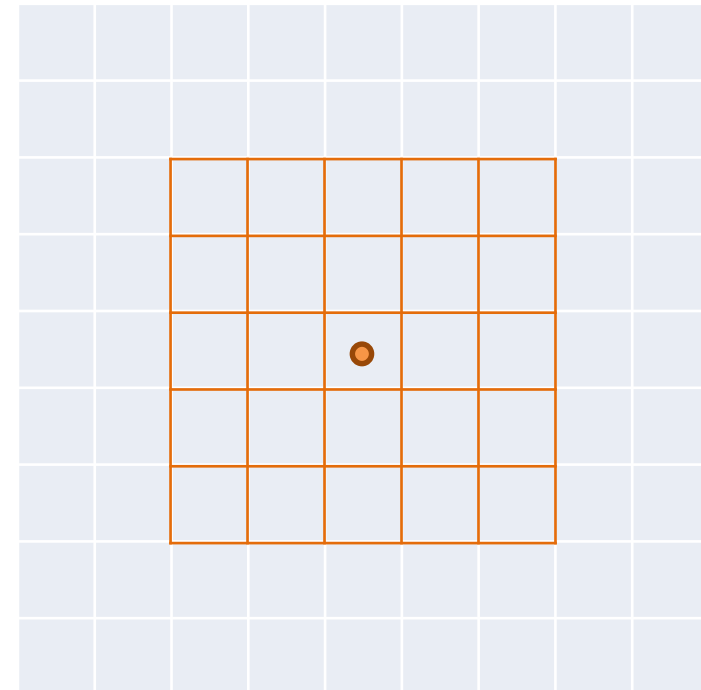


The receptive field of a neuron is defined as the region in the input volume that the neuron is looking at (i.e. is affected by).

The receptive field in Layer 1 is 5x5.
How about in Layer 2 and Layer 3?
9x9 and 13x13 respectively

Can we make the receptive field grow more quickly?

1st convolution layer
2nd convolution layer



Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

Stride=2
Padding=0

*

1	-1	0
-1	0	0
1	0	1

3x3

=

Strided convolution can be used to increase the receptive field in subsequent layers

Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

Stride=2

Padding=0

*

1	-1	0
-1	0	0
1	0	1

3x3

=

4		

Strided convolution can be used to increase the receptive field in subsequent layers

Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

Stride=2

Padding=0

*

1	-1	0
-1	0	0
1	0	1

3x3

=

4	3	

Strided convolution can be used to increase the receptive field in subsequent layers

Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

Stride=2

Padding=0

*

1	-1	0
-1	0	0
1	0	1

3x3

=

4	3	0

Strided convolution can be used to increase the receptive field in subsequent layers

Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

Stride=2

Padding=0

*

1	-1	0
-1	0	0
1	0	1

3x3

=

4	3	0
3		

Strided convolution can be used to increase the receptive field in subsequent layers

Strided Convolution

1	0	2	1	1	2	2
1	0	1	0	1	0	1
2	1	2	1	1	1	1
-1	0	1	2	1	2	2
1	2	0	-1	0	1	-1
6	1	7	-1	0	1	0
1	1	1	2	1	0	2

7x7

Stride=2

Padding=0

*

1	-1	0
-1	0	0
1	0	1

3x3

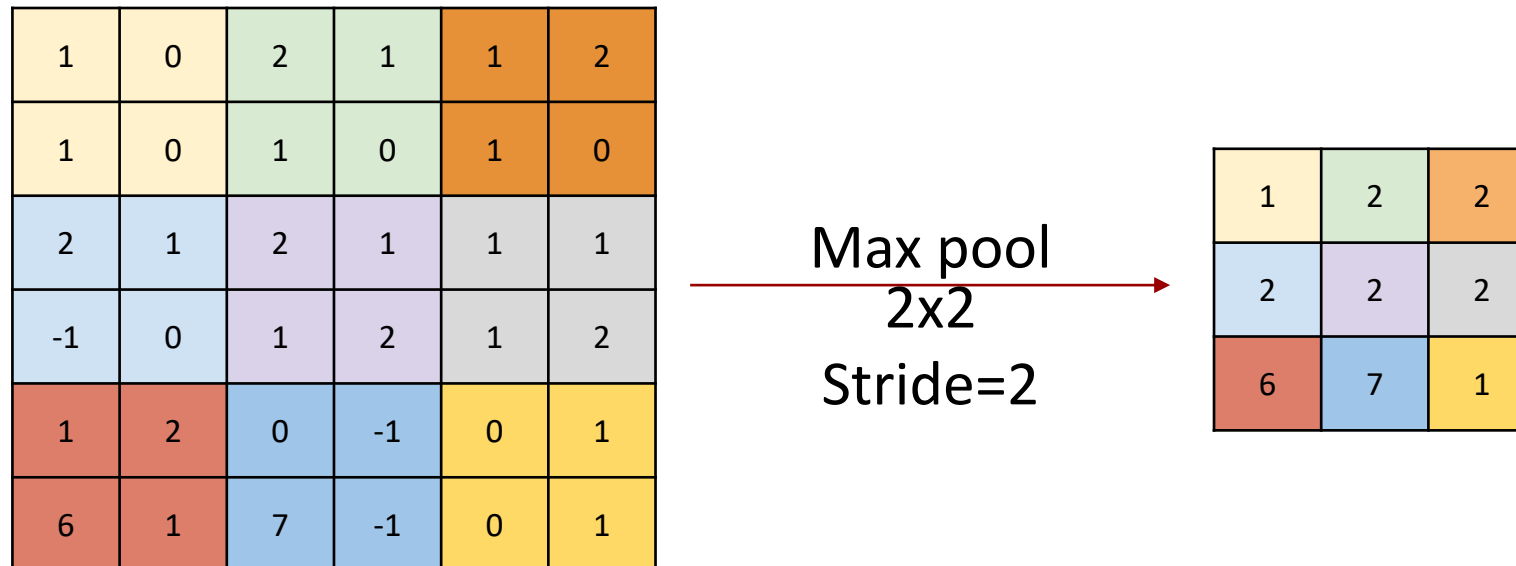
=

4	3	0
3	0	-2
-5	-4	2

Strided convolution can be used to increase the receptive field in subsequent layers

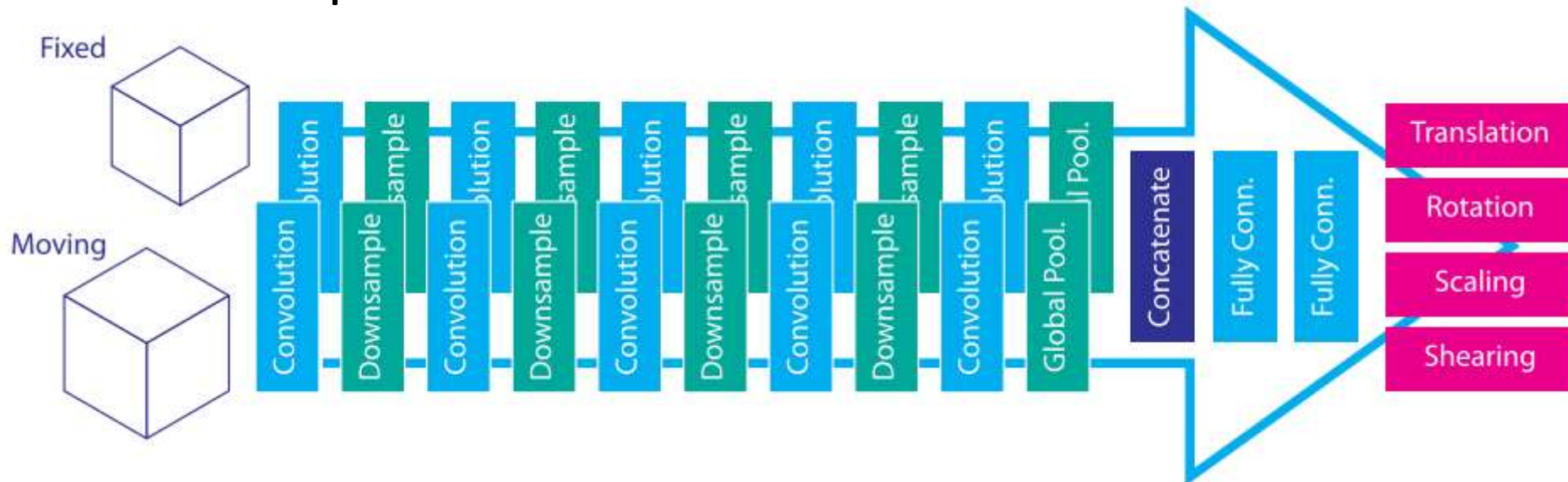
Pooling Layer

- **Pooling layers** downsample their input
 - Increases receptive field with a manageable number of parameters and computational cost
- A pooling layer is defined by
 1. Pooling size
 2. Pooling type (Maximum, Average, Stochastic)
 3. Stride



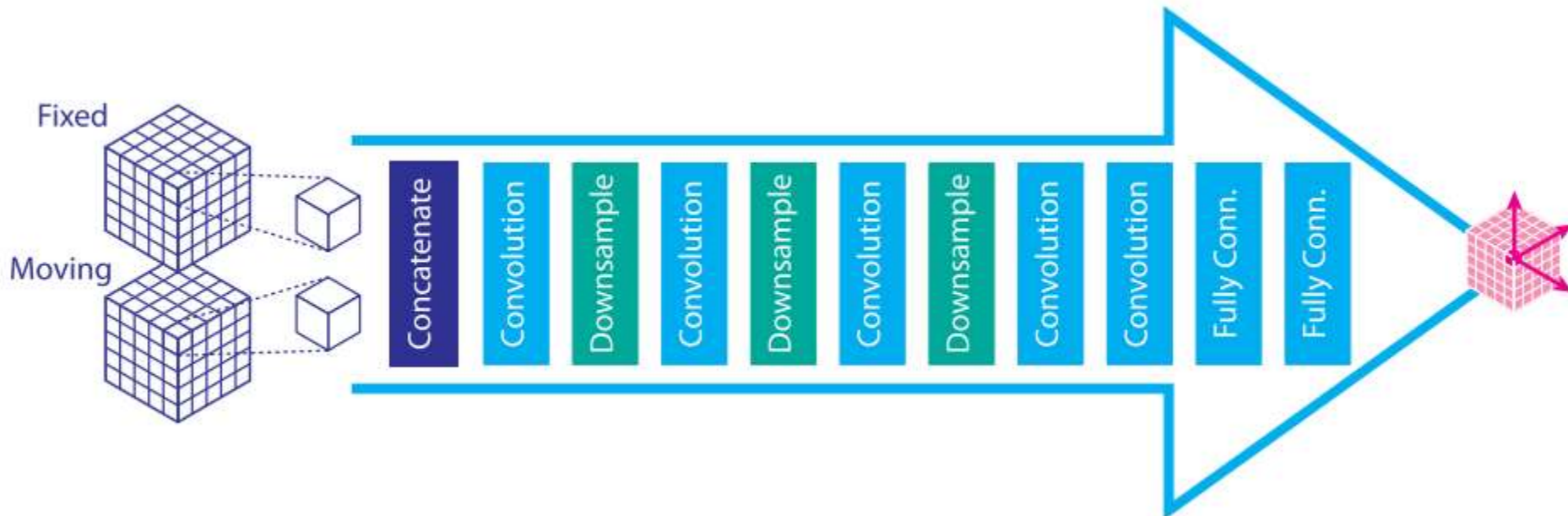
CNN-based Affine Registration

- For **affine registration**, [de Vos et al. 2019] initially process images separately and fuse after global pooling to permit image pairs with different sizes / resolutions
 - 3D images processed via 3D convolutions
 - Repeated downsampling (via average pooling or strided convolution) increases receptive field



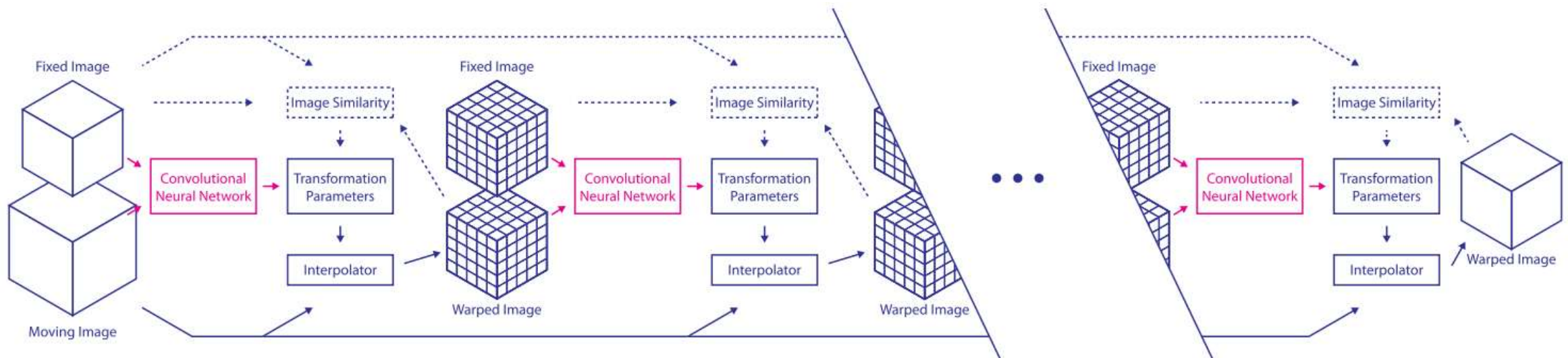
CNN-based Deformable Registration

- For **deformable registration**, [de Vos et al. 2019] resample both images to the same grid (based on affine parameters) and process them jointly to predict displacement vector field
 - Number of downsampling layers depends on desired grid resolution
 - Receptive field covers spatial support of B-splines



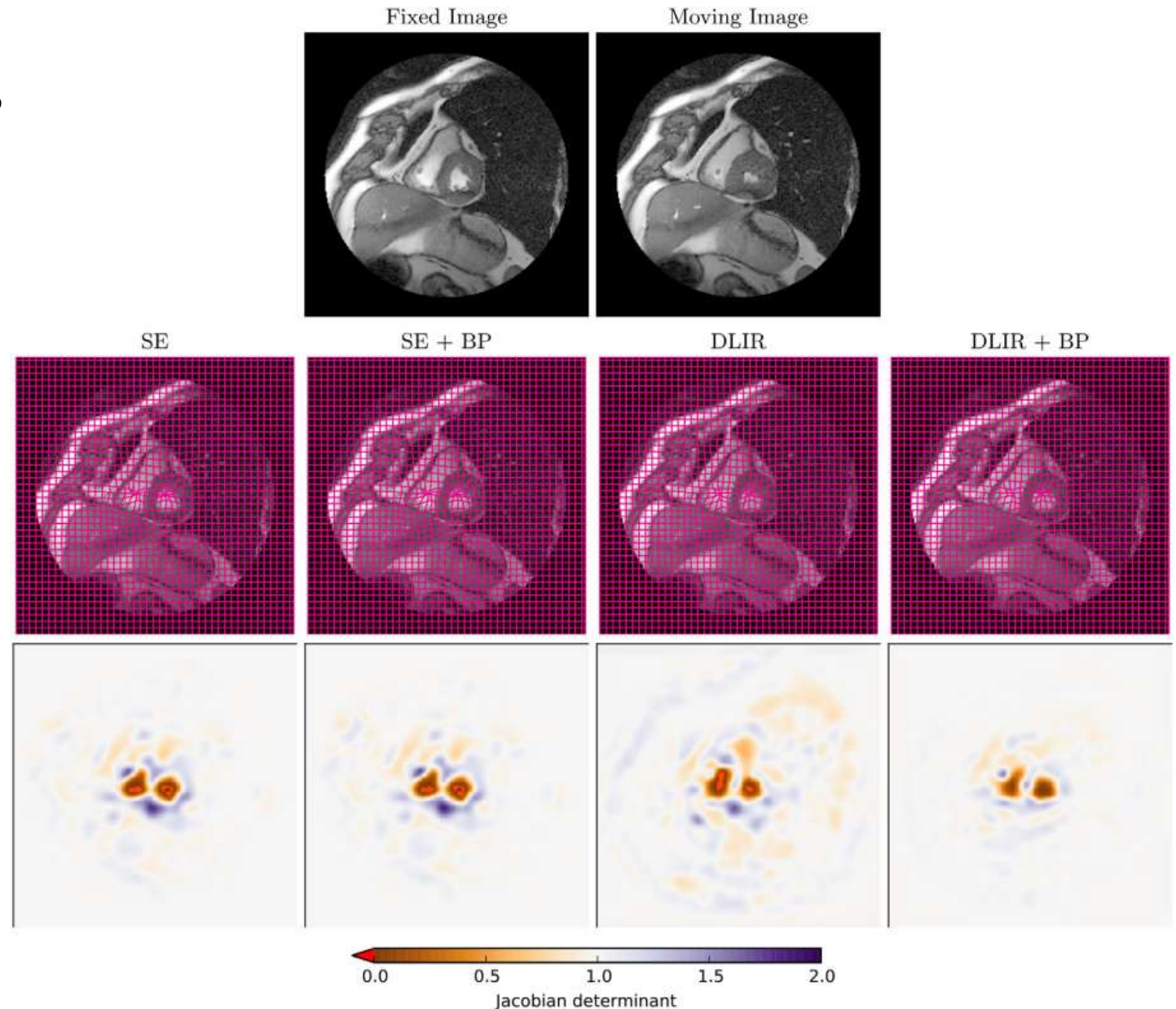
Hierarchical Multi-Stage Registration

- To obtain **fine-scale registration**, [de Vos et al. 2019] recommend a multi-step coarse-to-fine strategy
 - Affine registration (if needed), followed by multiple, increasingly fine deformable registrations
 - Each stage further refines the result of the previous one



Example Result: Learned Image Registration

- *Example:* End-diastolic vs end-systolic heart MR
 - SE = SimpleElastix (traditional toolbox)
 - DLIR = Deep Learning Image Registration
 - BP= Bending Penalty
- Two-stage DLIR was 350 times faster than single stage conventional



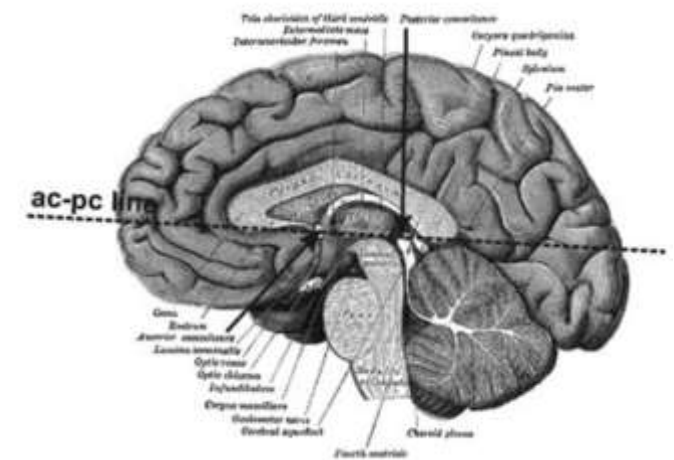
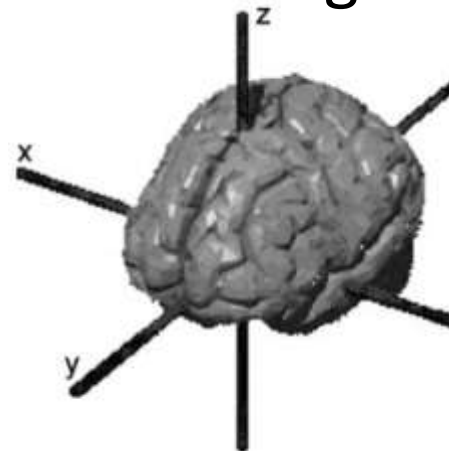
Summary: Neural Network based Image Registration

- **Neural networks** can achieve fast and accurate registration
- We considered a simple CNN-based **unsupervised** approach
 - Optimization of **cost function** analogous to classical registration
 - Affine, followed by (multi-stage) deformable registration
 - Strided convolutions / pooling for increasing receptive field
- Outlook:
 - **Supervised approaches** use image pairs with known transformations
 - **Diffeomorphic registration** also possible (requires integration layer)
 - **Transformer architectures** have an unlimited receptive field

4.9 Template Construction

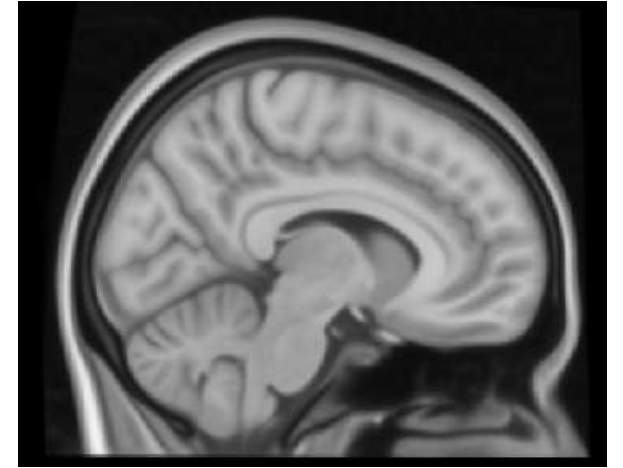
Talairach Space

- Talairach and Tournoux (1988) have defined a standardized brain coordinate system based on anatomical landmarks
 - Mid-sagittal plane defines yz-plane
 - y axis connects anterior and posterior commissure
 - Anterior commissure defines origin
 - Talairach atlas based on photos of one single brain
 - Linearly mapping to atlas yields “stereotactic coordinates”



MNI Template

- The Montreal Neurological Institute (MNI) has constructed widely used brain templates
 - Averaged 241 normal MRI brain scans after linear alignment to Talairach space, based on manually defined landmarks
 - **MNI305**: Linearly mapped 305 normal brains to the average from the previous step and averaged them
 - **ICBM152**: Linearly mapped 152 normal brains to MNI305; standard used by International Consortium for Brain Mapping (ICBM)
 - *Cave*: Coordinates diverge from Talairach!



Slice from ICBM152

Building Your Own Template

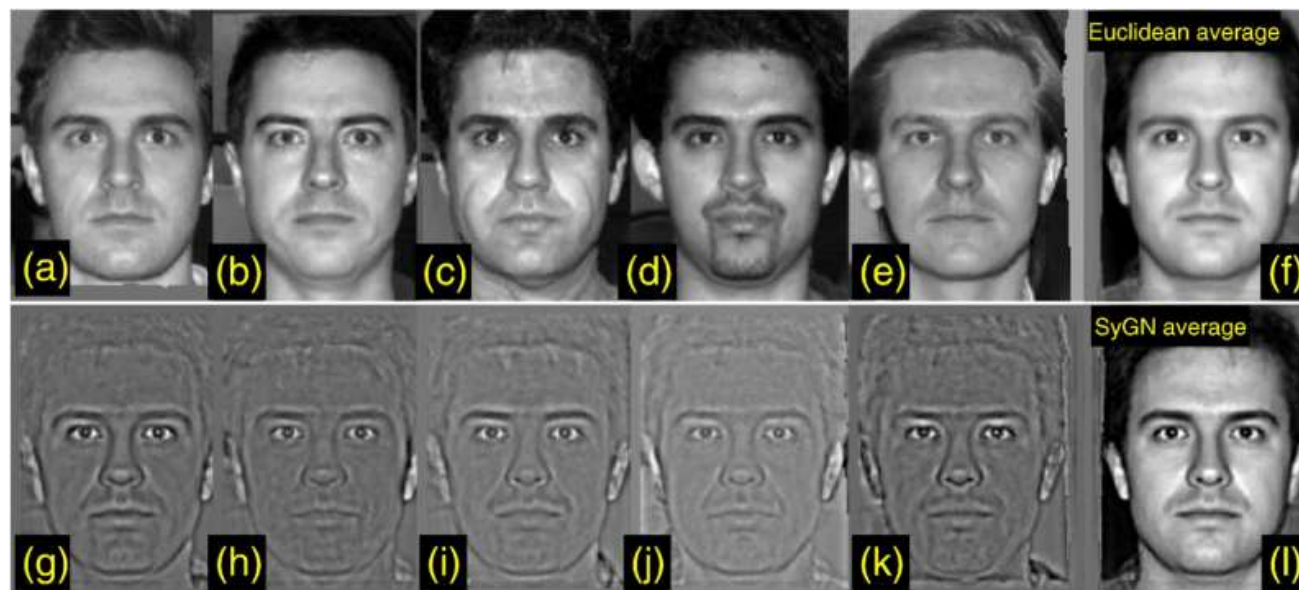
- For some applications (e.g., when studying specific age groups or disease), it is preferable to have a custom template built from your own data
 - **Simple idea:** Pick a representative individual, register all other data sets to it and average
 - Works reasonably well in many cases
 - **Questions:**
 - Can we avoid the selection bias?
 - Can we do better than simple averaging?

Iterative Template Refinement

- Symmetric Group-Wise Normalization (SyGN):
 - Initialize template as average image after affine registration
 - Iterate until convergence:
 1. Register all subjects to the template
 2. Update appearance of the template
 3. Update shape of the template
 - All subjects contribute equally to the update

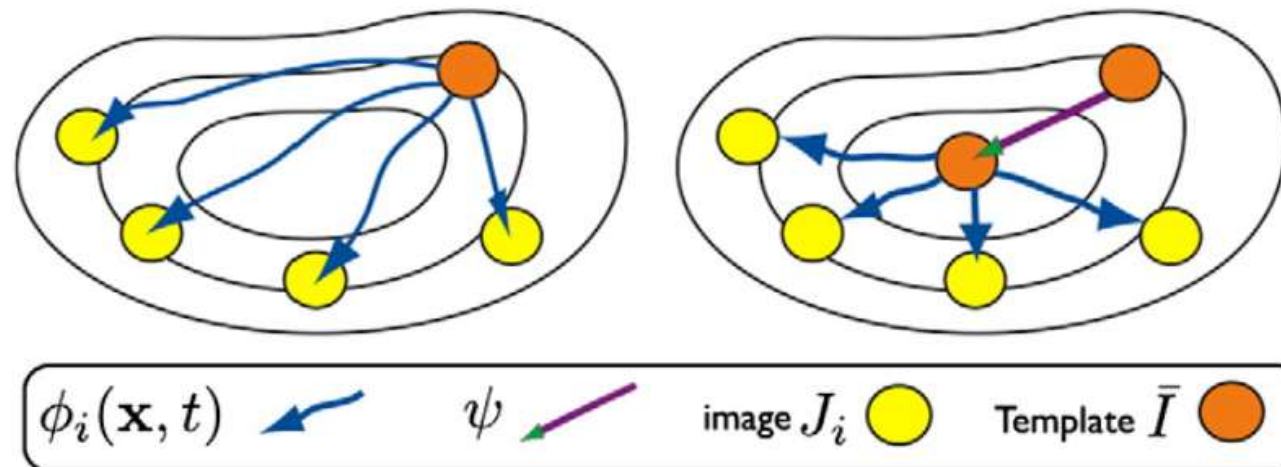
Appearance Updates

- **Goal:** Minimize the overall dissimilarity between all registered images and template
 - For L2 error, this amounts to simple averaging
 - For local cross-correlation, iterate:
 1. For all images, compute gradients of S^{LNC}
 2. Slightly smooth and average them
 3. Add a fraction of the result to the template



Shape Updates

- **Goal:** Minimize the overall deformation required to register all images to template
 - “Factor out” a mutual part of the deformation and apply it to the template
 - Achieved by averaging the respective velocity fields



Summary: Template Construction

You should know...

- about Talairach and MNI templates
- how to build your own template
- how to use templates for labeling

Further Reading

- Isaac N. Bankman: *Handbook of Medical Imaging. Processing and Analysis*. Academic Press, 2000
- A. Sotiras et al.: *Deformable Medical Image Registration: A survey*. IEEE Trans. Medical Imaging 32(7):1153-1190, 2013
- J. Chen et al.: *A survey on deep learning in medical image registration: New technologies, uncertainty, evaluation metrics, and beyond*. Medical Image Analysis 100:103385, 2025
- W.H. Press, S.A. Teukolsky, W.T. Vetterling, B.P. Flannery: *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, 3rd edition, 2007

Further Further Reading

- M. Jenkinson, S. Smith: *A global optimisation method for robust affine registration of brain images*. Medical Image Analysis 5:143-156, 2001
- B.B. Avants et al.: *Symmetric diffeomorphic image registration with cross-correlation: Evaluating automated labeling of elderly and neurodegenerative brain*. Medical Image Analysis 12:26-41, 2008
- B.B. Avants et al.: *The optimal template effect in hippocampus studies of diseased populations*. NeuroImage 49:2457-2466, 2010
- B.D. de Vos et al.: *A deep learning framework for unsupervised affine and deformable image registration*. Medical Image Analysis 52:128-143, 2019

Exam-Like Questions

1. Affine transformations in three-dimensional space have twelve degrees of freedom. What types of geometric transformations do they enable?
2. Normalized Mutual Information (NMI) can be defined as

$$NMI(X, Y) = \frac{H(X) + H(Y)}{H(X, Y)}$$

What do $H(X)$, $H(Y)$, and $H(X, Y)$ in this equation stand for? Why can discontinuities arise in their computation, and how can you address them? In which case would you consider to use NMI as a cost function for image registration? What algorithm could be used to maximize it?

3. How are nonlinear image transformations regularized in elastic registration?